

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN – ĐIỆN TỬ



BÁO CÁO THỰC TẬP AI
BÀI 6: SEMANTIC SEGMENTATION

MÔN HỌC: THỰC HÀNH HỌC MÁY VÀ TRÍ TUỆ NHÂN TẠO
GIẢNG VIÊN HƯỚNG DẪN: PGS_TS. TRƯƠNG NGỌC SƠN
SINH VIÊN THỰC HIỆN: TRƯƠNG PHÚC THỊNH - 23161336

Tp. Hồ Chí Minh, tháng 6 năm 2026

MỤC LỤC

1. GIỚI THIỆU	1
1.1 Tổng quan bài toán Semantic Segmentation	1
1.2 Mục tiêu bài thực hành	1
1.3 Dataset sử dụng	2
2. CƠ SỞ LÝ THUYẾT	3
2.1 Semantic Segmentation	3
2.2 Kiến trúc U-Net	4
2.3 Loss Function	6
2.4 Chỉ số đánh giá IoU và Dice Score	8
3. QUY TRÌNH THỰC HIỆN.....	10
3.1 Môi trường và thư viện.....	10
3.2 Chuẩn bị dữ liệu và tiền xử lý	11
3.3 Xây dựng mô hình và hàm đánh giá.....	12
3.4 Huấn luyện mô hình	13
3.5 Các tham số và công cụ sử dụng:	14
4. KẾT QUẢ THỰC NGHIỆM.....	16
4.1 Khám phá dataset và hiển thị ảnh mẫu.....	16
4.2 Chia dữ liệu, tạo DataLoader và kiểm tra dữ liệu đầu vào.....	19
4.3 Bài 7: Huấn luyện mô hình U-Net với BCEWithLogitsLoss.....	23
4.4 Bài 8: Thử nghiệm Dice Loss.....	28
4.5 Bài 9: Tính IoU trên tập validation.....	32
4.6 Bài 10: Hiển thị segmentation mask dự đoán.....	32
4.7 Bài 11: So sánh kết quả khi thay đổi learning rate	34
4.8 Bài 12: Thử nghiệm các batch size khác nhau	36
4.9 Bài 13: Phân tích các trường hợp mô hình dự đoán sai.....	37

4.10 Bài 14: Đề xuất cách cải thiện mô hình.....	39
4.11 Bài 15: Thử nghiệm một kiến trúc segmentation khác	39
4.12 Tổng hợp kết quả thực nghiệm.....	43
5. PHÂN TÍCH VÀ THẢO LUẬN.....	44
5.1 Nhận xét về kết quả đạt được	44
5.2 So sánh loss function	44
5.3 Ảnh hưởng của siêu tham số	44
5.4 Nguyên nhân lỗi dự đoán.....	44
5.5 Ưu điểm và hạn chế của mô hình	45
6. KẾT LUẬN VÀ HƯỚNG CẢI THIỆN.....	45

DANH MỤC HÌNH ẢNH

Hình 1: Pipeline thực hiện Semantic Segmentation	1
Hình 2: Ảnh gốc và segmentation mask trong Oxford-IIIT Pet.....	3
Hình 3: Hình ảnh mô tả kiến trúc Unet	6
Hình 4: Hình ảnh minh họa cho Loss function trong semantic segmentation	8
Hình 5. Minh họa cách tính chỉ số Intersection over Union (IoU).....	9
Hình 6. Minh họa cách tính chỉ số Dice Coefficient (DSC).....	10
Hình 7. Dataset the oxford-IIIT Pet Dataset trong bài	11
Hình 8. Sơ đồ khối mô hình unet và quy trình đánh giá trên tập Validation	13
Hình 9. Kiểm tra số lượng ảnh và mask của tập dataset	17
Hình 10: Minh họa ảnh gốc và segmentation mask trong bộ dữ liệu Oxford-IIIT Pet .	19
Hình 11: Chia tập train và validation trên dataset	20
Hình 12: Kết quả kiểm tra một batch dữ liệu sau khi tạo DataLoader	21
Hình 13: Kết quả kiểm tra mô hình U-Net sau khi xây dựng	22
Hình 14: Kết quả Huấn luyện mô hình U-Net với BCEWithLogitsLoss.....	26
Hình 15: Biểu đồ Loss, IoU và Dice Score khi huấn luyện U-Net với BCE Loss.....	27
Hình 16: Kết quả thử nghiệm Dice Loss	30
Hình 17: So sánh IoU và Dice Score giữa BCE Loss và Dice Loss	31
Hình 18: Kết quả dự đoán segmentation trên tập validation	34
Hình 19: Kết quả khi thay đổi learning rate	36
Hình 20: Kết quả khi thay đổi batchsize	37
Hình 21: Các trường hợp dự đoán có IoU thấp	38
Hình 22: Kết quả huấn luyện SegNet.....	41

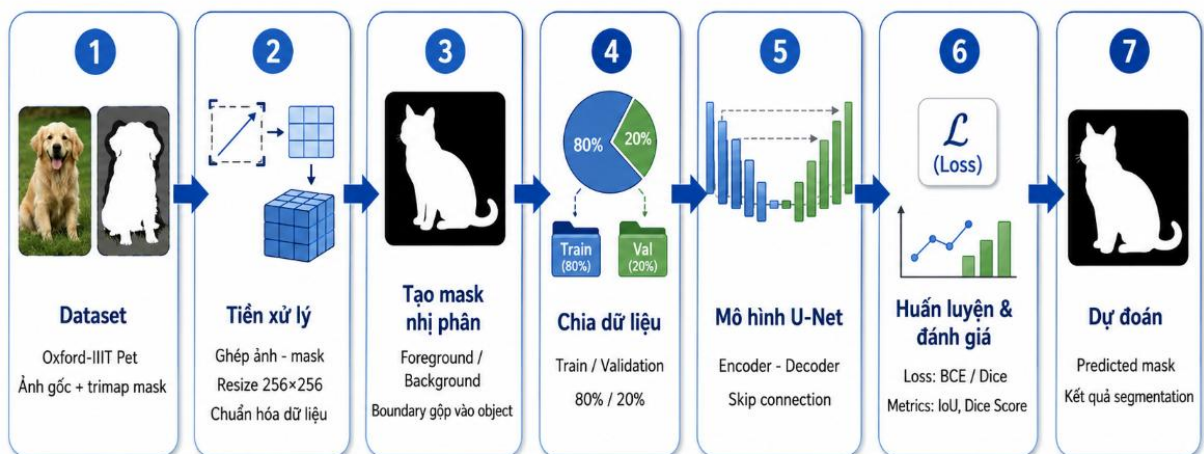
1. GIỚI THIỆU

1.1 Tổng quan bài toán Semantic Segmentation

Semantic Segmentation là bài toán gán nhãn cho từng pixel trong ảnh. Nếu phân loại ảnh chỉ trả lời ảnh thuộc lớp nào, còn object detection xác định vùng chứa đối tượng bằng bounding box, thì semantic segmentation tạo ra một bản đồ nhãn ở cùng độ phân giải với ảnh đầu vào. Cách biểu diễn này giúp mô hình nhận biết chính xác vùng thuộc đối tượng và vùng nền, phù hợp với các bài toán cần thông tin không gian chi tiết.

Trong bài thực hành số 6, mô hình U-Net được xây dựng bằng PyTorch để phân đoạn ảnh thú cưng trên bộ dữ liệu Oxford-IIIT Pet. Đầu vào của mô hình là ảnh RGB kích thước 256x256, đầu ra là mask nhị phân cho biết pixel thuộc đối tượng thú cưng hay nền.

Pipeline thực hiện Semantic Segmentation với Oxford-IIIT Pet



Hình: Pipeline xử lý dữ liệu, huấn luyện và dự đoán cho bài toán semantic segmentation

Hình 1: Pipeline thực hiện Semantic Segmentation

1.2 Mục tiêu bài thực hành

- Hiểu bản chất của semantic segmentation và sự khác biệt so với classification/object detection.
- Khám phá cấu trúc dataset gồm ảnh gốc và segmentation mask tương ứng.

- Xây dựng Dataset class, DataLoader và pipeline tiền xử lý trong PyTorch.
- Xây dựng, huấn luyện và đánh giá mô hình U-Net cho phân đoạn ảnh.
- Đánh giá mô hình bằng IoU và Dice Score, đồng thời trực quan hóa predicted mask.
- Thử nghiệm ảnh hưởng của loss function, learning rate và batch size đến kết quả.
- Phân tích các trường hợp dự đoán sai và đề xuất hướng cải thiện mô hình.

1.3 Dataset sử dụng

Bộ dữ liệu được sử dụng trong bài thực hành là **Oxford-IIIT Pet Dataset**, một bộ dữ liệu phổ biến trong các bài toán phân loại ảnh và phân đoạn ảnh. Dataset này bao gồm các ảnh thú cưng thuộc nhiều giống chó, mèo khác nhau, đi kèm với các mask tương ứng dùng để xác định vùng đối tượng trong ảnh. Trong notebook, dữ liệu được sử dụng gồm 7390 ảnh gốc và 7390 segmentation mask, trong đó mỗi ảnh có một mask tương ứng cùng tên.

Mask gốc của Oxford-IIIT Pet được biểu diễn dưới dạng trimap, gồm ba vùng chính: vùng đối tượng, vùng nền và vùng biên của đối tượng. Tuy nhiên, trong bài thực hành này, bài toán được đơn giản hóa thành phân đoạn nhị phân. Cụ thể, vùng đối tượng và vùng biên được gộp thành lớp foreground, đại diện cho vùng thú cưng cần phân đoạn; còn vùng nền được giữ lại là lớp background. Sau bước xử lý này, mỗi pixel trong mask chỉ thuộc một trong hai lớp: foreground hoặc background.

Việc chuyển mask về dạng nhị phân giúp mô hình U-Net tập trung vào nhiệm vụ chính là tách vùng thú cưng ra khỏi nền ảnh. Ảnh đầu vào và mask được resize về kích thước **256×256** để đảm bảo đồng nhất kích thước dữ liệu trước khi đưa vào mô hình huấn luyện.

Bảng 1: Cấu trúc và thông kê dữ liệu

Thành phần	Giá trị/thiết lập	Ý nghĩa
Dataset	Oxford-IIIT Pet	Tập ảnh thú cưng có ảnh RGB và trimap

		mask đi kèm.
Số ảnh	7390	Tổng số ảnh JPG được đọc từ thư mục images.
Số mask	7390	Mỗi ảnh có một mask PNG cùng tên trong thư mục annotations/trimaps.
Train/Validation	5912 / 1478	Chia 80% huấn luyện và 20% validation với seed = 42.
Kích thước ảnh	256 x 256	Resize ảnh và mask để đưa vào U-Net.
Mask nhị phân	0 hoặc 1	0 là background, 1 là object/boundary.



Hình 2: Ảnh gốc và segmentation mask trong Oxford-IIIT Pet

2. CƠ SỞ LÝ THUYẾT

2.1 Semantic Segmentation

Semantic Segmentation là bài toán gán nhãn cho từng pixel trong ảnh, nhằm xác định chính xác vùng thuộc đối tượng và vùng nền. Khác với classification chỉ phân loại toàn bộ ảnh, semantic segmentation tạo ra một mask có kích thước tương ứng với ảnh đầu vào, trong đó mỗi pixel được dự đoán thuộc một lớp nhất định.

Trong bài thực hành này, bài toán được đưa về dạng phân đoạn nhị phân với hai lớp: **foreground** là vùng thú cưng và **background** là phần nền. Mô hình sử dụng một kênh đầu ra để tạo mask dự đoán. Các giá trị logits được đưa qua hàm sigmoid để chuyển thành xác suất, sau đó dùng ngưỡng 0.5 để xác định pixel thuộc foreground hay background.

Khó khăn của semantic segmentation là mô hình vừa phải hiểu được ngữ nghĩa của đối tượng, vừa phải giữ lại thông tin vị trí và đường biên. Do đó, kiến trúc encoder-decoder có skip connection như U-Net rất phù hợp, vì nó giúp mô hình trích xuất đặc trưng sâu đồng thời khôi phục chi tiết không gian khi tạo mask đầu ra.

2.2 Kiến trúc U-Net

U-Net gồm hai nhánh chính. Nhánh encoder dùng các khối convolution và pooling để trích xuất đặc trưng ở nhiều mức độ. Nhánh decoder dùng transpose convolution để khôi phục độ phân giải. Các skip connection nối đặc trưng từ encoder sang decoder giúp giữ lại thông tin vị trí và biên của đối tượng.

Bảng 2: Cấu hình mô hình U-Net

Khối	Thiết lập trong notebook	Vai trò
DoubleConv	Conv2d + BatchNorm + ReLU lặp 2 lần	Học đặc trưng cục bộ ổn định hơn sau mỗi mức.
Encoder	Features: 32, 64, 128, 256	Giảm dần kích thước không gian, tăng số kênh đặc trưng.
Bottleneck	DoubleConv 256 -> 512	Biểu diễn đặc trưng sâu nhất của ảnh.
Decoder	ConvTranspose2d + skip connection	Tăng kích thước không gian và ghép thông tin từ encoder.
Output	Conv2d 1x1, 1 channel	Sinh mask nhị phân dưới dạng logits.
Số tham số	7.765.985	Toàn bộ tham số đều train được.

Về mặt tính toán, các lớp convolution trong U-Net làm thay đổi kích thước feature map theo công thức:

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - K}{S} \right\rfloor + 1$$

Trong đó (H_{in}) là kích thước đầu vào, (K) là kích thước kernel, (P) là padding và (S) là stride. Với các lớp convolution sử dụng kernel(3×3), padding bằng 1 và stride bằng 1, kích thước không gian của feature map được giữ nguyên sau mỗi phép tích chập. Sau đó, pooling làm giảm kích thước không gian, còn ConvTranspose2d ở decoder giúp tăng kích thước feature map để khôi phục lại độ phân giải ban đầu.

Skip connection trong U-Net có thể biểu diễn dưới dạng:

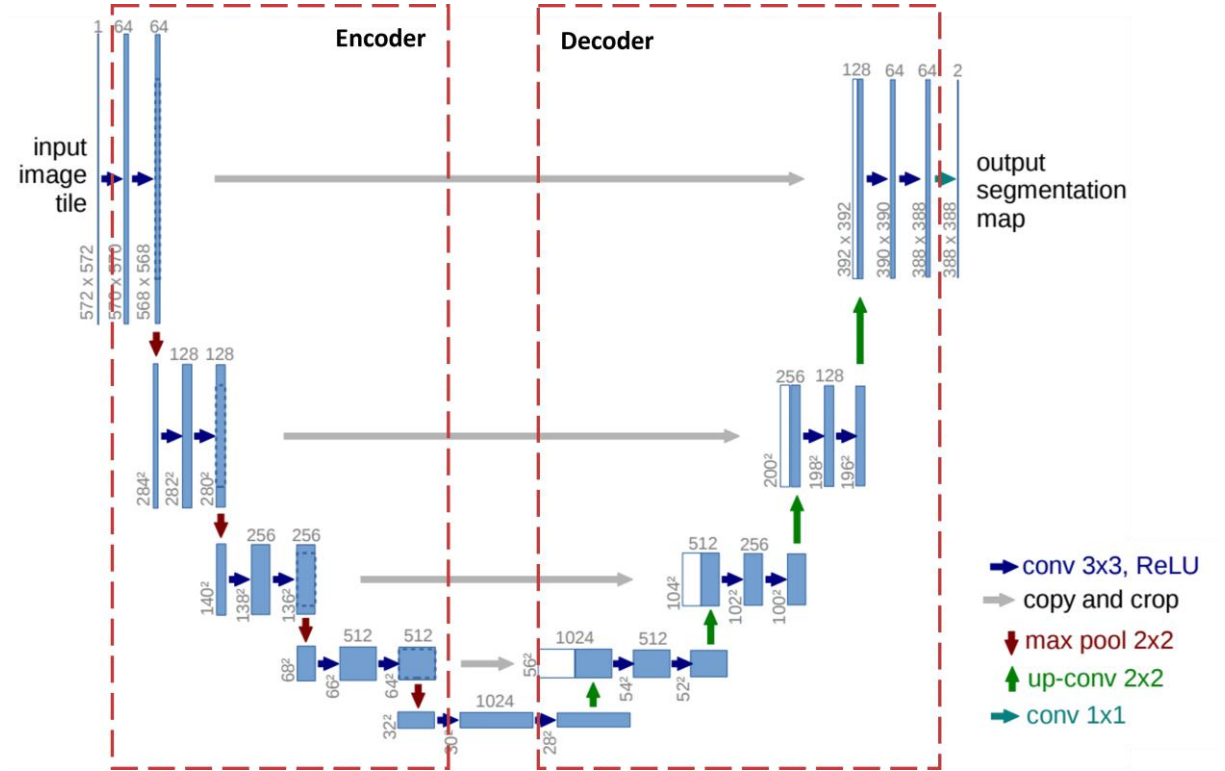
$$X_{decoder} = Concat(Up(X_{decoder}), X_{encoder})$$

Trong đó ($Up(X_{decoder})$) là feature map sau khi được tăng kích thước ở nhánh decoder, còn ($X_{encoder}$) là feature map tương ứng từ nhánh encoder. Việc ghép hai đặc trưng này giúp mô hình kết hợp thông tin ngữ nghĩa sâu với thông tin vị trí và đường biên của đối tượng.

Ở lớp đầu ra, U-Net sinh ra logits cho từng pixel. Các logits này được chuyển thành xác suất bằng hàm sigmoid:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Nếu ($p > 0.5$), pixel được dự đoán là foreground; ngược lại, pixel được xem là background.



Hình 3: Hình ảnh mô tả kiến trúc Unet

2.3 Loss Function

Trong bài toán semantic segmentation nhị phân, mô hình cần dự đoán mỗi pixel thuộc lớp foreground hay background. Vì vậy, hàm mất mát được sử dụng phải đánh giá được mức độ sai khác giữa mask dự đoán và mask ground truth. Trong bài thực hành này, hai hàm loss chính được sử dụng và so sánh là BCEWithLogitsLoss và Dice Loss.

BCEWithLogitsLoss là hàm mất mát phù hợp cho bài toán phân loại nhị phân theo từng pixel. Hàm này kết hợp trực tiếp hai bước gồm sigmoid và Binary Cross Entropy trong cùng một phép tính. Điều này giúp quá trình huấn luyện ổn định hơn so với việc tự áp dụng sigmoid rồi mới tính BCE. Với mỗi pixel, mô hình sinh ra một giá trị logits, sau đó hàm loss sẽ so sánh giá trị dự đoán với nhãn thật của pixel đó.

Binary Cross Entropy có thể được biểu diễn như sau:

$$BCE = -[y \log(p) + (1 - y) \log(1 - p)]$$

Trong đó, (y) là nhãn thật của pixel, với $(y = 1)$ là foreground và $(y = 0)$ là background; (p) là xác suất pixel thuộc foreground sau khi đưa logits qua hàm sigmoid. Giá trị BCE càng nhỏ cho thấy dự đoán của mô hình càng gần với ground truth.

Bên cạnh BCEWithLogitsLoss, bài thực hành còn thử nghiệm **Dice Loss**. Dice Loss được xây dựng dựa trên Dice Score, một chỉ số đo mức độ chồng lấp giữa mask dự đoán và mask thật. Dice Score được tính theo công thức:

$$Dice = \frac{2 \times Intersection}{Prediction + GroundTruth}$$

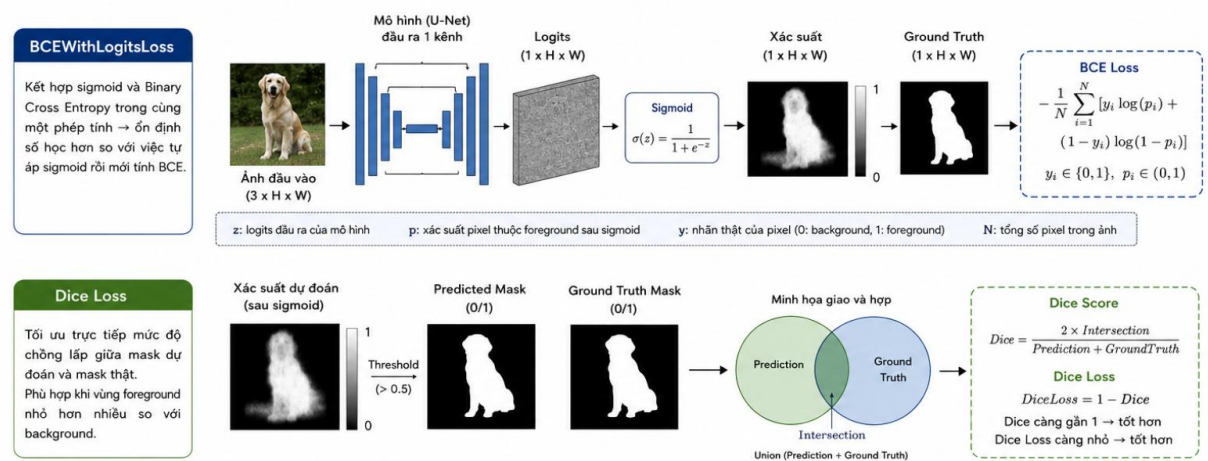
Trong đó, (Intersection) là phần giao giữa predicted mask và ground truth mask; (Prediction) là tổng số pixel được mô hình dự đoán là foreground; còn (GroundTruth) là tổng số pixel foreground trong mask thật. Dice Score càng gần 1 thì mask dự đoán càng giống với mask thật.

Dice Loss được định nghĩa như sau:

$$DiceLoss = 1 - Dice$$

Do đó, Dice Loss càng nhỏ thì chất lượng phân đoạn càng tốt. Ưu điểm của Dice Loss là tối ưu trực tiếp mức độ chồng lấp giữa hai mask, nên thường phù hợp với các bài toán segmentation, đặc biệt khi vùng foreground nhỏ hơn nhiều so với background. Tuy nhiên, trong quá trình thực nghiệm, Dice Loss có thể dao động mạnh hơn BCE, tùy thuộc vào dữ liệu, số epoch và cách khởi tạo mô hình.

Việc so sánh BCEWithLogitsLoss và Dice Loss giúp đánh giá hàm mất mát nào phù hợp hơn với bài toán phân đoạn ảnh thú cưng trong cấu hình huấn luyện hiện tại. BCEWithLogitsLoss tập trung vào độ chính xác phân loại từng pixel, trong khi Dice Loss tập trung vào mức độ trùng khớp tổng thể giữa mask dự đoán và mask ground truth.



Hình 4: Hình ảnh minh họa cho Loss function trong semantic segmentation

2.4 Chỉ số đánh giá IoU và Dice Score

Trong bài toán semantic segmentation, việc đánh giá mô hình không chỉ dựa vào loss mà còn cần sử dụng các chỉ số đo mức độ trùng khớp giữa **predicted mask** và **ground truth mask**. Hai chỉ số phổ biến được sử dụng trong bài thực hành này là **Intersection over Union (IoU)** và **Dice Score**.

Intersection over Union (IoU) là chỉ số đo tỉ lệ giữa phần giao và phần hợp của mask dự đoán với mask thật. Phần giao là vùng pixel mà mô hình dự đoán đúng thuộc đối tượng, còn phần hợp là toàn bộ vùng pixel thuộc đối tượng trong predicted mask hoặc ground truth mask. Công thức IoU được biểu diễn như sau:

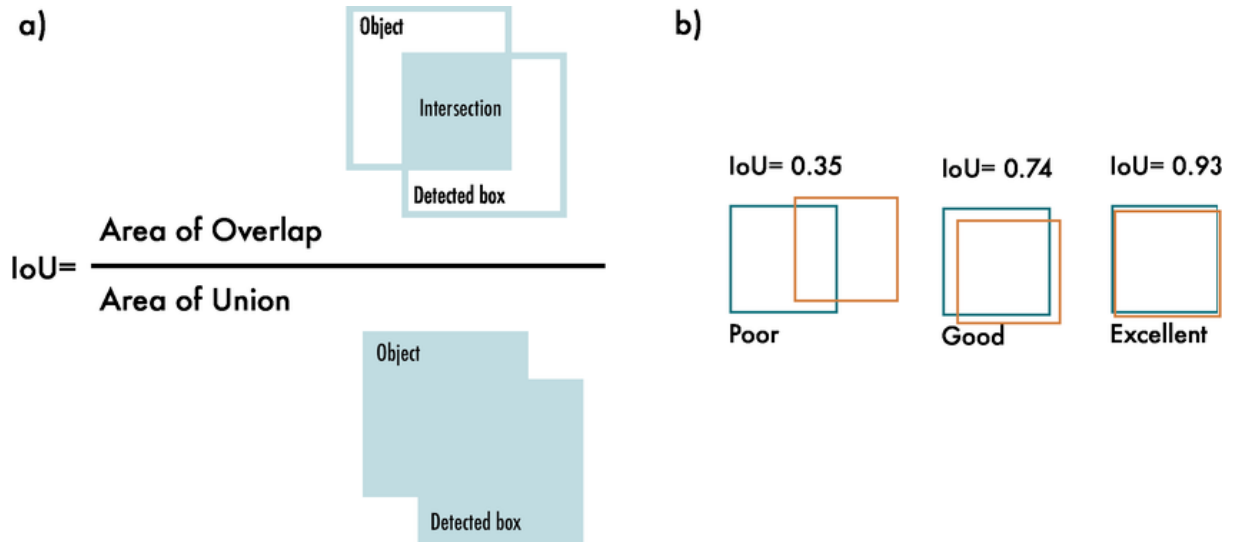
$$IoU = \frac{Intersection}{Union}$$

Trong đó:

$$Intersection = Prediction \cap GroundTruth$$

$$Union = Prediction \cup GroundTruth$$

Giá trị IoU nằm trong khoảng từ 0 đến 1. IoU càng gần 1 cho thấy vùng dự đoán của mô hình càng trùng khớp với mask thật. Ngược lại, IoU thấp cho thấy mô hình dự đoán sai nhiều, có thể do bỏ sót đối tượng, dự đoán thừa vùng nền hoặc xác định sai đường biên.

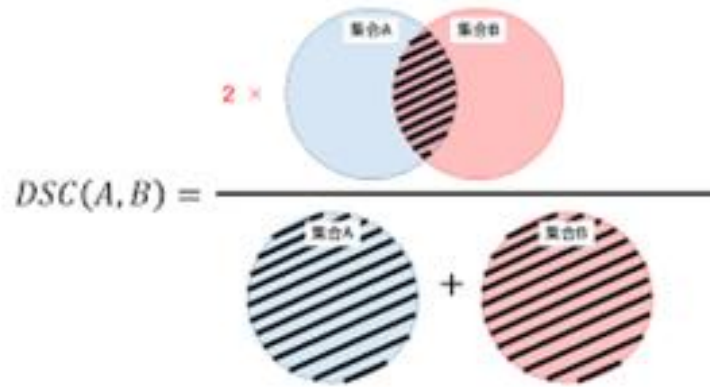


Hình 5. Minh họa cách tính chỉ số Intersection over Union (IoU)

Bên cạnh IoU, **Dice Score** cũng được sử dụng để đánh giá mức độ chồng lấp giữa predicted mask và ground truth mask. Dice Score được tính theo công thức:

$$Dice = \frac{2 \times Intersection}{Prediction + GroundTruth}$$

Trong đó, (Prediction) là tổng số pixel được mô hình dự đoán là foreground, còn (GroundTruth) là tổng số pixel foreground trong mask thật. Dice Score càng cao thì mask dự đoán càng giống với mask ground truth. Chỉ số này thường nhạy hơn trong các bài toán có vùng foreground nhỏ, vì nó tập trung nhiều vào mức độ trùng khớp giữa hai vùng mask.



Hình 6. Minh họa cách tính chỉ số Dice Core (DSC)

Trong notebook, đầu ra của mô hình U-Net là logits. Các logits này được đưa qua hàm sigmoid để chuyển thành xác suất từng pixel thuộc foreground. Sau đó, ngưỡng 0.5 được sử dụng để tạo mask nhị phân: nếu xác suất lớn hơn 0.5 thì pixel được xem là foreground, ngược lại là background. Sau khi có mask nhị phân, IoU và Dice Score được tính cho từng batch, sau đó lấy giá trị trung bình trên toàn bộ tập validation. Nhờ đó, có thể đánh giá tổng quát chất lượng phân đoạn của mô hình trên dữ liệu chưa được dùng trong quá trình huấn luyện.

3. QUY TRÌNH THỰC HIỆN

3.1 Môi trường và thư viện

Notebook được chạy trên môi trường Kaggle có GPU NVIDIA RTX PRO 6000 Blackwell Server Edition. Các thư viện chính gồm PyTorch, torchvision, PIL, numpy, matplotlib và tqdm. Seed được cố định bằng 42 để kết quả chia dữ liệu và khởi tạo có tính lặp lại tốt hơn.

Bảng 3: Tham số huấn luyện chính

Nhóm	Thiết lập
Thiết bị	CUDA available: True; GPU: NVIDIA RTX PRO 6000 Blackwell Server Edition
Framework	PyTorch, torchvision

Optimizer	Adam
Image size	256 x 256
Batch size chính	8
Epoch chính	20 epoch với BCE Loss; 10 epoch với Dice Loss
Learning rate chính	0.001

3.2 Chuẩn bị dữ liệu và tiền xử lý

The Oxford-IIIT Pet Dataset

Dataset of 37 categories of dogs and cats



[Data Card](#)
[Code \(95\)](#)
[Discussion \(0\)](#)
[Suggestions \(0\)](#)

About Dataset

Context

The Oxford-IIIT Pet Dataset is a 37 category pet dataset with roughly 100 images for each class created by the Visual Geometry Group at Oxford. The images have large variations in scale, pose and lighting. All images have an associated ground truth annotation of the breed, head ROI, and pixel level trimap segmentation.

Insiration

Usability ⓘ
8.75

License
[CC0: Public Domain](#)

Expected update frequency
Annually

Hình 7. Dataset the oxford-IIIT Pet Dataset trong bài

- Đọc danh sách ảnh JPG trong thư mục images và mask PNG trong thư mục trimaps.
- Kiểm tra số lượng ảnh và mask tương ứng đều bằng 7390.
- Resize ảnh và mask về kích thước 256x256. Với mask, dùng interpolation NEAREST để không sinh giá trị lớp mới.
- Chuyển ảnh sang tensor float32 có kích thước [3, 256, 256].
- Chuyển trimap thành mask nhị phân: giá trị khác background được xem là foreground.
- Chia dataset thành train và validation theo tỉ lệ 80/20.

3.3 Xây dựng mô hình và hàm đánh giá

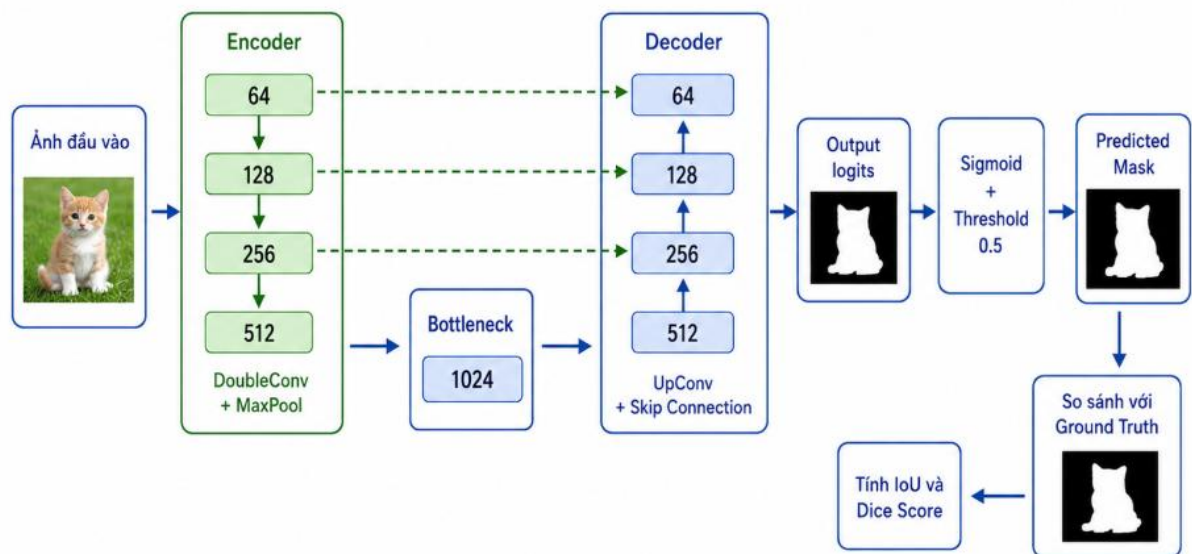
Sau khi hoàn thành bước chuẩn bị dữ liệu và tạo `DataLoader`, mô hình U-Net được xây dựng bằng PyTorch để thực hiện nhiệm vụ phân đoạn ảnh. Trong notebook, mô hình được thiết kế gồm hai thành phần chính là **DoubleConv** và **UNet**.

Khối **DoubleConv** là khối tích chập cơ bản của mô hình. Mỗi khối gồm hai lớp `Conv2d` liên tiếp, kết hợp với `BatchNorm2d` và hàm kích hoạt `ReLU`. Việc sử dụng hai lớp convolution giúp mô hình học được đặc trưng cục bộ tốt hơn tại mỗi mức, trong khi `BatchNorm` giúp ổn định phân phối dữ liệu đầu ra của các lớp, hỗ trợ quá trình huấn luyện hội tụ nhanh và ổn định hơn. Hàm `ReLU` được dùng để đưa tính phi tuyến vào mô hình, giúp mạng học được các đặc trưng phức tạp của ảnh.

Lớp **UNet** được xây dựng theo cấu trúc encoder-decoder. Ở nhánh encoder, ảnh đầu vào lần lượt đi qua các khối `DoubleConv` và lớp `MaxPooling` để giảm kích thước không gian, đồng thời tăng số kênh đặc trưng. Quá trình này giúp mô hình trích xuất các đặc trưng từ đơn giản đến phức tạp của đối tượng trong ảnh. Ở phần bottleneck, mô hình học biểu diễn đặc trưng sâu nhất trước khi chuyển sang nhánh decoder.

Trong nhánh decoder, mô hình sử dụng `ConvTranspose2d` để tăng kích thước feature map, khôi phục dần độ phân giải ban đầu. Tại mỗi mức decoder, feature map sau khi upsample được ghép với feature map tương ứng từ encoder thông qua skip connection. Các feature map từ encoder được lưu lại trong quá trình forward để sử dụng cho bước ghép này. Skip connection giúp mô hình giữ lại thông tin vị trí và đường biên của đối tượng, từ đó cải thiện chất lượng mask dự đoán.

Cuối cùng, mô hình sử dụng lớp `Conv2d` kích thước (1×1) để đưa số kênh đầu ra về 1. Đầu ra này là logits của từng pixel, sau đó được đưa qua hàm sigmoid khi tính toán mask dự đoán. Với bài toán phân đoạn nhị phân, mỗi pixel sẽ được dự đoán là thuộc vùng foreground hoặc background.



Hình 8. Sơ đồ khối mô hình unet và quy trình đánh giá trên tập Validation

Bên cạnh mô hình, các hàm đánh giá cũng được xây dựng để theo dõi chất lượng huấn luyện. Hai chỉ số được sử dụng là **IoU** và **Dice Score**. Trong quá trình đánh giá, logits đầu ra của mô hình được chuyển thành xác suất bằng sigmoid, sau đó dùng ngưỡng 0.5 để tạo mask nhị phân. Mask dự đoán được so sánh với ground truth mask để tính IoU và Dice Score. Các chỉ số này được tính trung bình trên từng batch và sau đó lấy trung bình trên toàn bộ tập validation, giúp đánh giá mức độ chính xác của mô hình trong việc phân đoạn vùng thú cưng.

3.4 Huấn luyện mô hình

Sau khi xây dựng mô hình U-Net và các hàm đánh giá, quá trình huấn luyện được thực hiện trên tập training và kiểm tra trên tập validation sau mỗi epoch. Trong notebook, quá trình này được chia thành hai hàm chính là `train_one_epoch` và `validate_one_epoch` nhằm giúp chương trình rõ ràng, dễ theo dõi và thuận tiện cho việc đánh giá kết quả.

Ở hàm `train_one_epoch`, mô hình được chuyển sang chế độ huấn luyện bằng `model.train()`. Với mỗi batch dữ liệu, ảnh đầu vào và mask tương ứng được đưa lên thiết bị tính toán, sau đó mô hình thực hiện lan truyền thuận để tạo ra mask dự đoán. Kết quả dự đoán được so sánh với ground truth mask thông qua hàm loss. Sau

đó, optimizer sẽ xóa gradient cũ, thực hiện lan truyền ngược để tính gradient mới và cập nhật trọng số mô hình bằng thuật toán Adam.

Ở hàm `validate_one_epoch`, mô hình được chuyển sang chế độ đánh giá bằng `model.eval()`. Trong giai đoạn này, mô hình chỉ thực hiện dự đoán và tính toán các chỉ số loss, IoU và Dice Score trên tập validation, không cập nhật trọng số. Việc sử dụng `torch.no_grad()` giúp giảm bộ nhớ sử dụng và tăng tốc quá trình đánh giá.

Sau mỗi epoch, các giá trị train loss, validation loss, IoU và Dice Score được lưu lại để theo dõi quá trình học của mô hình. Notebook cũng thực hiện lưu checkpoint sau từng epoch và lưu lại **best model** khi chỉ số validation IoU đạt giá trị cao hơn các epoch trước đó. Cách làm này giúp đảm bảo mô hình tốt nhất không bị mất, đồng thời có thể tiếp tục huấn luyện lại nếu quá trình chạy bị gián đoạn.

3.5 Các tham số và công cụ sử dụng:

Trong quá trình thực hiện bài thực hành, các tham số huấn luyện được thiết lập nhằm đảm bảo mô hình U-Net có thể học ổn định và phù hợp với khả năng tính toán của môi trường Kaggle. Ảnh đầu vào và mask được resize về cùng kích thước **256×256** trước khi đưa vào mô hình. Batch size chính được chọn là **8**, giúp cân bằng giữa tốc độ huấn luyện và dung lượng bộ nhớ GPU.

Mô hình được huấn luyện bằng thuật toán tối ưu **Adam** với learning rate ban đầu là **0.001**. Đây là giá trị learning rate phổ biến trong các bài toán deep learning, giúp mô hình hội tụ tương đối ổn định. Hàm mất mát chính được sử dụng là **BCEWithLogitsLoss**, đồng thời bài thực hành cũng thử nghiệm thêm **Dice Loss** để so sánh hiệu quả giữa các hàm loss trong bài toán segmentation.

Đối với cấu hình chính, mô hình U-Net được huấn luyện trong **20 epoch** với BCEWithLogitsLoss. Ngoài ra, Dice Loss được thử nghiệm trong **10 epoch** để đánh giá khả năng tối ưu mức độ chồng lấp giữa predicted mask và ground truth mask. Các

chỉ số đánh giá chính bao gồm **IoU** và **Dice Score**, được tính trên tập validation sau mỗi epoch.

Bảng 4: Các tham số huấn luyện chính

Tham số	Giá trị sử dụng	Ý nghĩa
Dataset	Oxford-IIIT Pet	Bộ dữ liệu ảnh thú cưng và segmentation mask
Image size	256×256	Kích thước ảnh và mask sau khi resize
Train/Validation	80% / 20%	Tỉ lệ chia dữ liệu huấn luyện và kiểm tra
Batch size chính	8	Số mẫu xử lý trong mỗi lần cập nhật
Optimizer	Adam	Thuật toán cập nhật trọng số mô hình
Learning rate chính	0.001	Tốc độ học của mô hình
Loss chính	BCEWithLogitsLoss	Hàm mất mát cho phân đoạn nhị phân
Loss thử nghiệm	Dice Loss	Hàm loss tối ưu mức độ chồng lấp mask
Epoch BCE Loss	20	Số vòng huấn luyện với BCEWithLogitsLoss
Epoch Dice Loss	10	Số vòng huấn luyện với Dice Loss
Metrics	IoU, Dice Score	Chỉ số đánh giá chất lượng phân đoạn
Threshold	0.5	Ngưỡng chuyển xác suất thành mask nhị phân
Framework	PyTorch	Thư viện xây dựng và huấn luyện mô hình
Thiết bị	CUDA/GPU	Tăng tốc quá trình huấn luyện

Công cụ và thư viện sử dụng

Ngoài các tham số huấn luyện, bài thực hành còn sử dụng một số công cụ và thư viện hỗ trợ cho quá trình xây dựng mô hình, xử lý dữ liệu, huấn luyện và trực quan hóa kết quả. Môi trường thực hiện chính là **Kaggle Notebook**, có hỗ trợ GPU để tăng tốc quá trình huấn luyện mô hình deep learning.

Ngôn ngữ lập trình được sử dụng là **Python**. Mô hình U-Net được xây dựng và huấn luyện bằng thư viện **PyTorch**, kết hợp với **torchvision** để hỗ trợ các bước biến đổi dữ liệu như resize ảnh, chuyển ảnh sang tensor và tiền xử lý đầu vào. Thư viện **PIL/Pillow** được dùng để đọc ảnh và mask, trong khi **NumPy** hỗ trợ xử lý mảng dữ liệu, đặc biệt trong quá trình chuyển đổi segmentation mask về dạng nhị phân.

4. KẾT QUẢ THỰC NGHIỆM

Chương này trình bày kết quả thực nghiệm của bài Lab 6 theo thứ tự các bài tập từ giai đoạn huấn luyện mô hình đến đánh giá, so sánh tham số và phân tích lỗi. Các kết quả chính được tổng hợp bằng bảng số liệu, biểu đồ và hình ảnh dự đoán segmentation mask trên tập validation.

4.1 Khám phá dataset và hiển thị ảnh mẫu

Trong bài thực hành này, bộ dữ liệu được sử dụng là **Oxford-IIIT Pet Dataset**. Dữ liệu gồm hai thành phần chính: thư mục chứa ảnh gốc và thư mục chứa segmentation mask tương ứng. Ảnh gốc được lưu dưới định dạng `.jpg`, còn mask được lưu dưới định dạng `.png` trong thư mục `annotations/trimaps`.

Ở bước đầu tiên, chương trình đọc toàn bộ danh sách ảnh trong thư mục `images` và danh sách mask trong thư mục `trimaps`. Các file không hợp lệ hoặc file ẩn bắt đầu bằng ký tự `._` được loại bỏ để tránh lỗi trong quá trình đọc dữ liệu. Sau khi kiểm tra, kết quả cho thấy dataset có **7390 ảnh gốc** và **7390 mask**. Số lượng ảnh và mask bằng nhau chứng tỏ mỗi ảnh đều có một mask tương ứng, đảm bảo dữ liệu phù hợp để xây dựng bài toán semantic segmentation.

```
import os
IMAGE_DIR = "/kaggle/input/datasets/thinhtruong2334/the-oxford/images/images"
MASK_DIR = "/kaggle/input/datasets/thinhtruong2334/the-oxford/annotations/annotations/trimaps"
```

```

image_files = sorted([
    f for f in os.listdir(IMAGE_DIR)
    if f.endswith(".jpg")
])

mask_files = sorted([
    f for f in os.listdir(MASK_DIR)
    if f.endswith(".png")
    and not f.startswith("._")
])

print("Số ảnh:", len(image_files))
print("Số mask:", len(mask_files))

```

Kết quả:

Số ảnh: 7390
Số mask: 7390

Hình 9. Kiểm tra số lượng ảnh và mask của tập dataset

Nhận xét:

Kết quả kiểm tra cho thấy dataset có cấu trúc đầy đủ, gồm 7390 ảnh và 7390 mask tương ứng. Ảnh gốc và mask mẫu được hiển thị đúng cặp, cho thấy dữ liệu có thể sử dụng cho các bước tiền xử lý, chia tập train/validation và huấn luyện mô hình U-Net ở các bước tiếp theo

```

from PIL import Image
import matplotlib.pyplot as plt
import os

idx = 13

```

```
image_path = os.path.join(IMAGE_DIR, image_files[idx])
mask_path = os.path.join(MASK_DIR, mask_files[idx])

image = Image.open(image_path).convert("RGB")
mask = Image.open(mask_path)

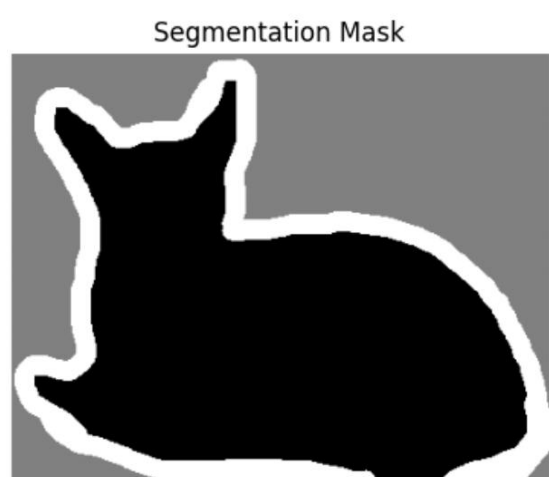
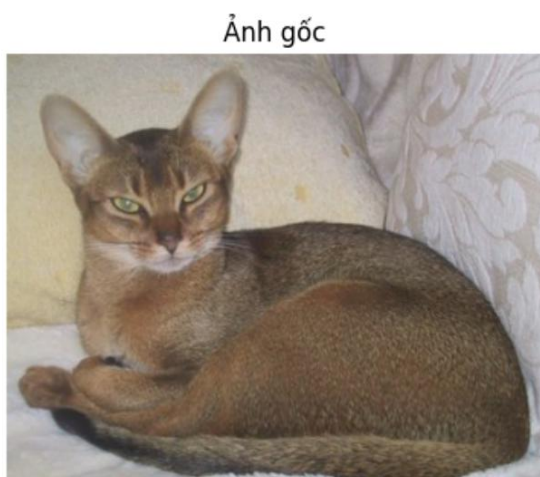
plt.figure(figsize=(10,4))

plt.subplot(1,2,1)
plt.imshow(image)
plt.title("Ảnh gốc")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(mask, cmap="gray")
plt.title("Segmentation Mask")
plt.axis("off")

plt.show()
```

Tiếp theo, một ảnh mẫu trong dataset được hiển thị cùng với segmentation mask tương ứng. Ảnh gốc là ảnh RGB của thú cưng, còn mask là ảnh biểu diễn vùng phân đoạn của đối tượng. Trong mask, các vùng khác nhau thể hiện đối tượng, nền và vùng biên. Việc hiển thị đồng thời ảnh gốc và mask giúp kiểm tra trực quan xem dữ liệu có được ghép đúng cặp hay không, đồng thời giúp hiểu rõ hơn đầu vào và đầu ra mong muốn của mô hình.



Hình 10: Minh họa ảnh gốc và segmentation mask trong bộ dữ liệu Oxford-IIIT Pet

Qua kết quả quan sát, ảnh gốc và segmentation mask có sự tương ứng rõ ràng. Mask thể hiện vùng đối tượng thú cưng trong ảnh, đây là cơ sở để mô hình học cách phân biệt vùng foreground và background. Bước khám phá dữ liệu này rất quan trọng vì nếu ảnh và mask không khớp nhau, mô hình sẽ học sai và kết quả phân đoạn không chính xác..

4.2 Chia dữ liệu, tạo DataLoader và kiểm tra dữ liệu đầu vào

Sau khi kiểm tra cấu trúc dataset, toàn bộ dữ liệu được đưa vào lớp PetSegmentationDataset. Lớp này có nhiệm vụ đọc ảnh gốc và mask tương ứng, resize về kích thước **256×256**, chuyển ảnh sang tensor và chuyển mask về dạng nhị phân. Trong bài toán này, mask nhị phân chỉ gồm hai giá trị: **0** đại diện cho background và **1** đại diện cho foreground.

```
from torch.utils.data import random_split

full_dataset = PetSegmentationDataset(
    image_files=image_files,
    image_dir=IMAGE_DIR,
    mask_dir=MASK_DIR,
    image_size=256
)

train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size

train_dataset, val_dataset = random_split(
    full_dataset,
    [train_size, val_size],
    generator=torch.Generator().manual_seed(SEED)
```

```
)  
  
print("Train:", len(train_dataset))  
print("Validation:", len(val_dataset))
```

Dataset sau đó được chia thành hai phần: tập huấn luyện và tập validation. Tỉ lệ chia dữ liệu là **80% cho training** và **20% cho validation**. Với tổng số 7390 mẫu, kết quả chia dữ liệu thu được là **5912 ảnh cho tập train** và **1478 ảnh cho tập validation**. Việc chia dữ liệu theo tỉ lệ này giúp mô hình có đủ dữ liệu để học, đồng thời vẫn có một tập riêng để đánh giá khả năng tổng quát hóa sau mỗi epoch.

Train: 5912
Validation: 1478

Hình 11: Chia tập train và validation trên dataset

Sau khi chia dữ liệu, DataLoader được tạo cho cả tập train và validation. Batch size được sử dụng là **8**, nghĩa là mỗi lần huấn luyện mô hình sẽ xử lý 8 ảnh cùng lúc. Với `train_loader`, tham số `shuffle=True` được sử dụng để xáo trộn dữ liệu trong quá trình huấn luyện, giúp mô hình tránh học theo thứ tự cố định của ảnh. Với `val_loader`, `shuffle=False` được sử dụng để giữ nguyên thứ tự dữ liệu khi đánh giá.

```
BATCH_SIZE = 8  
  
train_loader = DataLoader(  
    train_dataset,  
    batch_size=BATCH_SIZE,  
    shuffle=True,  
    num_workers=2,
```



```

        pin_memory=True
    )

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=2,
    pin_memory=True
)

# Kiểm tra dữ liệu
images, masks = next(iter(train_loader))
print("Image batch:", images.shape)
print("Mask batch :", masks.shape)
print("Image dtype:", images.dtype)
print("Mask dtype :", masks.dtype)
print("Mask values:", torch.unique(masks))

```

```

Image batch: torch.Size([8, 3, 256, 256])
Mask batch : torch.Size([8, 1, 256, 256])
Image dtype: torch.float32
Mask dtype : torch.float32
Mask values: tensor([0., 1.])

```

Hình 12: Kết quả kiểm tra một batch dữ liệu sau khi tạo DataLoader

Kết quả kiểm tra một batch dữ liệu cho thấy tensor ảnh có kích thước:

[8, 3, 256, 256]

Trong đó, 8 là số ảnh trong một batch, 3 là số kênh màu RGB, còn 256×256 là kích thước ảnh sau khi resize. Tensor mask có kích thước:

[8,1,256,256]

Trong đó, 1 là số kênh của mask nhị phân. Cả ảnh và mask đều có kiểu dữ liệu `torch.float32`, phù hợp để đưa vào mô hình huấn luyện. Ngoài ra, giá trị duy nhất trong mask là 0 và 1, chứng tỏ quá trình chuyển đổi mask sang dạng nhị phân đã được thực hiện đúng.

```
..   Tổng số tham số: 7765985
     Số tham số train được: 7765985
     Input shape : torch.Size([1, 3, 256, 256])
     Output shape: torch.Size([1, 1, 256, 256])
```

Hình 13: Kết quả kiểm tra mô hình U-Net sau khi xây dựng

Bên cạnh đó, mô hình U-Net cũng được kiểm tra bằng một ảnh đầu vào giả lập có kích thước:

[1,3,256,256]

Kết quả đầu ra của mô hình có kích thước:

[1,1,256,256]

Điều này cho thấy mô hình nhận ảnh RGB đầu vào và sinh ra một mask nhị phân một kênh có cùng kích thước không gian với ảnh ban đầu. Tổng số tham số của mô hình là **7.765.985**, và toàn bộ các tham số này đều được huấn luyện trong quá trình training.

Bảng 5: Kết quả kiểm tra dữ liệu và mô hình

Nội dung kiểm tra	Kết quả
-------------------	---------

Số ảnh gốc	7390
Số mask	7390
Tập train	5912
Tập validation	1478
Batch size	8
Shape ảnh trong batch	[8, 3, 256, 256]
Shape mask trong batch	[8, 1, 256, 256]
Kiểu dữ liệu ảnh	torch.float32
Kiểu dữ liệu mask	torch.float32
Giá trị mask	0 và 1
Input shape mô hình	[1, 3, 256, 256]
Output shape mô hình	[1, 1, 256, 256]
Tổng số tham số	7.765.985

Nhận xét:

Kết quả kiểm tra cho thấy quá trình chuẩn bị dữ liệu đã được thực hiện đúng. Ảnh và mask có kích thước phù hợp, mask đã được chuyển thành dạng nhị phân, DataLoader tạo batch đúng định dạng và mô hình U-Net sinh đầu ra đúng kích thước yêu cầu. Đây là điều kiện cần thiết để tiếp tục bước huấn luyện mô hình segmentation ở các phần sau.

4.3 Bài 7: Huấn luyện mô hình U-Net với BCEWithLogitsLoss

Ở bài tập này, mô hình U-Net được huấn luyện trên bộ dữ liệu Oxford-IIIT Pet sau khi ảnh và mask đã được resize về kích thước 256×256. Hàm mất mát chính được sử dụng là BCEWithLogitsLoss, optimizer là Adam với learning rate 0.001 và batch size chính là 8. Mô hình được huấn luyện trong 20 epoch, sau mỗi epoch tiến hành đánh giá trên tập validation bằng các chỉ số loss, IoU và Dice Score.

```
EPOCHS = 20
LR = 0.001

model_bce = UNet().to(device)
```

```

criterion_bce = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model_bce.parameters(), lr=LR)

history_bce = {
    "train_loss": [],
    "val_loss": [],
    "train_iou": [],
    "val_iou": [],
    "train_dice": [],
    "val_dice": []
}

best_iou = 0.0
start_epoch = 0

best_model_path = os.path.join(WORK_DIR, "best_unet_bce.pth")
checkpoint_path = os.path.join(WORK_DIR, "checkpoint_unet_bce.pth")

# Load checkpoint nếu đã có
if os.path.exists(checkpoint_path):
    checkpoint = torch.load(checkpoint_path, map_location=device)

    model_bce.load_state_dict(checkpoint["model_state_dict"])
    optimizer.load_state_dict(checkpoint["optimizer_state_dict"])

    history_bce = checkpoint["history"]
    best_iou = checkpoint["best_iou"]
    start_epoch = checkpoint["epoch"] + 1

    print("Đã load checkpoint.")
    print("Tiếp tục từ epoch:", start_epoch + 1)
    print("Best IoU hiện tại:", best_iou)

for epoch in range(start_epoch, EPOCHS):
    print(f"\nEpoch [{epoch+1}/{EPOCHS}]")

    train_loss, train_iou, train_dice = train_one_epoch(
        model_bce,
        train_loader,

```

```

        optimizer,
        criterion_bce
    )

    val_loss, val_iou, val_dice = validate_one_epoch(
        model_bce,
        val_loader,
        criterion_bce
    )

    history_bce["train_loss"].append(train_loss)
    history_bce["val_loss"].append(val_loss)
    history_bce["train_iou"].append(train_iou)
    history_bce["val_iou"].append(val_iou)
    history_bce["train_dice"].append(train_dice)
    history_bce["val_dice"].append(val_dice)

    print(f"Train Loss: {train_loss:.4f} | Train IoU: {train_iou:.4f} | Train Dice: {train_dice:.4f}")
    print(f"Val Loss: {val_loss:.4f} | Val IoU: {val_iou:.4f} | Val Dice: {val_dice:.4f}")

print("Hoàn tất training BCE Loss.")
print("Best Validation IoU:", best_iou)
print("Best model:", best_model_path)
print("Checkpoint:", checkpoint_path)

```

Kết quả:

Epoch [1/20] Train Loss: 0.4774 Train IoU: 0.5729 Train Dice: 0.7102 Val Loss: 0.4450 Val IoU: 0.6183 Val Dice: 0.7474 Đã lưu checkpoint epoch 1 Đã lưu best model mới! Epoch [2/20] Train Loss: 0.3660 Train IoU: 0.6758 Train Dice: 0.7940 Val Loss: 0.5124 Val IoU: 0.6164 Val Dice: 0.7416 Đã lưu checkpoint epoch 2 Epoch [3/20] Train Loss: 0.3150 Train IoU: 0.7197 Train Dice: 0.8254 Val Loss: 0.2709 Val IoU: 0.7543 Val Dice: 0.8502 Đã lưu checkpoint epoch 3 Đã lưu best model mới! Epoch [4/20] Train Loss: 0.2847 Train IoU: 0.7462 Train Dice: 0.8433 Val Loss: 0.2842 Val IoU: 0.7523 Val Dice: 0.8482 Đã lưu checkpoint epoch 4 Epoch [5/20] Train Loss: 0.2590 Train IoU: 0.7705 Train Dice: 0.8604 Val Loss: 0.2562 Val IoU: 0.7667 Val Dice: 0.8568 Đã lưu checkpoint epoch 5 Đã lưu best model mới!	Epoch [6/20] Train Loss: 0.2404 Train IoU: 0.7853 Train Dice: 0.8701 Val Loss: 0.2580 Val IoU: 0.7717 Val Dice: 0.8600 Đã lưu checkpoint epoch 6 Đã lưu best model mới! Epoch [7/20] Train Loss: 0.2266 Train IoU: 0.7986 Train Dice: 0.8789 Val Loss: 0.2138 Val IoU: 0.8027 Val Dice: 0.8820 Đã lưu checkpoint epoch 7 Đã lưu best model mới! Epoch [8/20] Train Loss: 0.2104 Train IoU: 0.8129 Train Dice: 0.8885 Val Loss: 0.2459 Val IoU: 0.7880 Val Dice: 0.8711 Đã lưu checkpoint epoch 8 Epoch [9/20] Train Loss: 0.2031 Train IoU: 0.8186 Train Dice: 0.8923 Val Loss: 0.2142 Val IoU: 0.8090 Val Dice: 0.8867 Đã lưu checkpoint epoch 9 Đã lưu best model mới! Epoch [10/20] Train Loss: 0.1896 Train IoU: 0.8295 Train Dice: 0.8996 Val Loss: 0.2164 Val IoU: 0.8034 Val Dice: 0.8830 Đã lưu checkpoint epoch 10
---	---

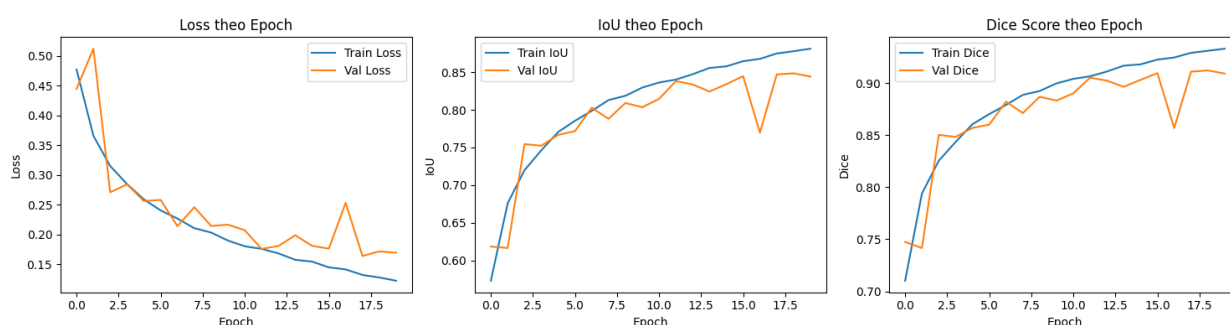
Epoch [11/20] Train Loss: 0.1800 Train IoU: 0.8362 Train Dice: 0.9039 Val Loss: 0.2070 Val IoU: 0.8144 Val Dice: 0.8900 Đã lưu checkpoint epoch 11 Đã lưu best model mới! Epoch [12/20] Train Loss: 0.1756 Train IoU: 0.8401 Train Dice: 0.9064 Val Loss: 0.1757 Val IoU: 0.8381 Val Dice: 0.9051 Đã lưu checkpoint epoch 12 Đã lưu best model mới! Epoch [13/20] Train Loss: 0.1680 Train IoU: 0.8472 Train Dice: 0.9110 Val Loss: 0.1804 Val IoU: 0.8337 Val Dice: 0.9022 Đã lưu checkpoint epoch 13 Epoch [14/20] Train Loss: 0.1573 Train IoU: 0.8556 Train Dice: 0.9166 Val Loss: 0.1986 Val IoU: 0.8243 Val Dice: 0.8963 Đã lưu checkpoint epoch 14 Epoch [15/20] Train Loss: 0.1542 Train IoU: 0.8577 Train Dice: 0.9179 Val Loss: 0.1808 Val IoU: 0.8340 Val Dice: 0.9030 Đã lưu checkpoint epoch 15	Epoch [16/20] Train Loss: 0.1446 Train IoU: 0.8646 Train Dice: 0.9224 Val Loss: 0.1760 Val IoU: 0.8446 Val Dice: 0.9094 Đã lưu checkpoint epoch 16 Đã lưu best model mới! Epoch [17/20] Train Loss: 0.1411 Train IoU: 0.8677 Train Dice: 0.9244 Val Loss: 0.2531 Val IoU: 0.7696 Val Dice: 0.8567 Đã lưu checkpoint epoch 17 Epoch [18/20] Train Loss: 0.1318 Train IoU: 0.8748 Train Dice: 0.9289 Val Loss: 0.1637 Val IoU: 0.8470 Val Dice: 0.9108 Đã lưu checkpoint epoch 18 Đã lưu best model mới! Epoch [19/20] Train Loss: 0.1275 Train IoU: 0.8778 Train Dice: 0.9309 Val Loss: 0.1714 Val IoU: 0.8484 Val Dice: 0.9120 Đã lưu checkpoint epoch 19 Đã lưu best model mới! Epoch [20/20] Train Loss: 0.1220 Train IoU: 0.8812 Train Dice: 0.9329 Val Loss: 0.1692 Val IoU: 0.8442 Val Dice: 0.9090 Đã lưu checkpoint epoch 20 Hoàn tất training BCE Loss. Best Validation IoU: 0.8484472770948668
--	---

Hình 14: Kết quả Huấn luyện mô hình U-Net với BCEWithLogitsLoss

Kết quả cho thấy train loss giảm dần qua các epoch, trong khi train IoU và train Dice tăng đều. Điều này chứng tỏ mô hình học được đặc trưng phân đoạn của vùng thú cưng. Trên tập validation, IoU đạt giá trị tốt nhất là 0.8484 ở epoch 19 và Dice Score tương ứng đạt 0.9120. Đây là kết quả khá tốt đối với mô hình U-Net tự xây dựng từ đầu trên ảnh kích thước 256×256 .

Bảng 6: Kết quả huấn luyện U-Net với BCEWithLogitsLoss

Epoch	Train Loss	Train IoU	Train Dice	Val Loss	Val IoU	Val Dice
1	0.4774	0.5729	0.7102	0.4450	0.6183	0.7474
3	0.3150	0.7197	0.8254	0.2709	0.7543	0.8502
5	0.2590	0.7705	0.8604	0.2562	0.7667	0.8568
10	0.1896	0.8295	0.8996	0.2164	0.8034	0.8830
15	0.1542	0.8577	0.9179	0.1808	0.8340	0.9030
18	0.1318	0.8748	0.9289	0.1637	0.8470	0.9108
19	0.1275	0.8778	0.9309	0.1714	0.8484	0.9120
20	0.1220	0.8812	0.9329	0.1692	0.8442	0.9090



Hình 15: Biểu đồ Loss, IoU và Dice Score khi huấn luyện U-Net với BCE Loss

Nhận xét:

Dựa vào Bảng 4.1 và Hình 4.1, có thể thấy mô hình U-Net khi huấn luyện với BCEWithLogitsLoss cho kết quả cải thiện rõ rệt qua các epoch. Train Loss giảm liên

tục từ 0.4774 ở epoch 1 xuống còn 0.1220 ở epoch 20, cho thấy mô hình học tốt trên tập huấn luyện. Đồng thời, Train IoU tăng từ 0.5729 lên 0.8812 và Train Dice tăng từ 0.7102 lên 0.9329, chứng tỏ khả năng phân đoạn vùng đối tượng ngày càng chính xác hơn.

Trên tập validation, Val IoU cũng tăng từ 0.6183 ở epoch 1 lên mức cao nhất 0.8484 tại epoch 19, trong khi Val Dice đạt giá trị tốt nhất 0.9120. Điều này cho thấy mô hình không chỉ học tốt trên tập train mà còn có khả năng tổng quát hóa tương đối tốt trên dữ liệu validation. Tuy nhiên, đường Val Loss và Val IoU có một vài dao động ở các epoch sau, đặc biệt quanh epoch 16–17. Nguyên nhân có thể do tập validation có nhiều ảnh khó, nền phức tạp hoặc vùng biên đối tượng không rõ ràng.

Nhìn chung, mô hình U-Net với BCEWithLogitsLoss đạt kết quả tốt cho bài toán phân đoạn ảnh thú cưng. Epoch 19 là epoch có kết quả validation tốt nhất với Val IoU = 0.8484 và Val Dice = 0.9120, vì vậy đây có thể được xem là mô hình tốt nhất trong quá trình huấn luyện.

4.4 Bài 8: Thử nghiệm Dice Loss

Bên cạnh BCEWithLogitsLoss, Dice Loss được thử nghiệm nhằm tối ưu trực tiếp mức độ chồng lấp giữa predicted mask và ground truth mask. Dice Loss phù hợp với bài toán segmentation vì nó đánh giá chất lượng dự đoán ở cấp độ vùng mask thay vì chỉ xét riêng từng pixel.

Code thực hiện:

```
EPOCHS_DICE = 10
LR_DICE = 0.001

model_dice = UNet().to(device)

criterion_dice = DiceLoss()
optimizer_dice = optim.Adam(model_dice.parameters(), lr=LR_DICE)

history_dice = {
    "train_loss": [],
    "val_loss": [],
```



```

        "train_iou": [],
        "val_iou": [],
        "train_dice": [],
        "val_dice": []
    }

best_iou_dice = 0
best_dice_model_path = os.path.join(WORK_DIR, "best_unet_dice.pth")

for epoch in range(EPOCHS_DICE):
    print(f"\nEpoch [{epoch+1}/{EPOCHS_DICE}]")

    train_loss, train_iou, train_dice_value = train_one_epoch(
        model_dice, train_loader, optimizer_dice, criterion_dice
    )
    val_loss, val_iou, val_dice_value = validate_one_epoch(
        model_dice, val_loader, criterion_dice
    )
    history_dice["train_loss"].append(train_loss)
    history_dice["val_loss"].append(val_loss)
    history_dice["train_iou"].append(train_iou)
    history_dice["val_iou"].append(val_iou)
    history_dice["train_dice"].append(train_dice_value)
    history_dice["val_dice"].append(val_dice_value)
    print(f"Train Loss: {train_loss:.4f} | Train IoU: {train_iou:.4f} | Train Dice: {train_dice_value:.4f}")
    print(f"Val Loss: {val_loss:.4f} | Val IoU: {val_iou:.4f} | Val Dice: {val_dice_value:.4f}")
    if val_iou > best_iou_dice:
        best_iou_dice = val_iou
        torch.save(model_dice.state_dict(), best_dice_model_path)
        print("Đã lưu model Dice Loss tốt nhất!")

print("Best Dice Loss Validation IoU:", best_iou_dice)

```

Kết quả:

Epoch [1/10] Train Loss: 0.2814 Train IoU: 0.5976 Train Dice: 0.7316 Val Loss: 0.2318 Val IoU: 0.6426 Val Dice: 0.7691 Đã lưu model Dice Loss tốt nhất! Epoch [2/10] Train Loss: 0.2134 Train IoU: 0.6674 Train Dice: 0.7872 Val Loss: 0.2408 Val IoU: 0.6330 Val Dice: 0.7595 Epoch [3/10] Train Loss: 0.1910 Train IoU: 0.6978 Train Dice: 0.8093 Val Loss: 0.1944 Val IoU: 0.6939 Val Dice: 0.8058 Đã lưu model Dice Loss tốt nhất! Epoch [4/10] Train Loss: 0.1732 Train IoU: 0.7222 Train Dice: 0.8270 Val Loss: 0.1876 Val IoU: 0.7049 Val Dice: 0.8125 Đã lưu model Dice Loss tốt nhất! Epoch [5/10] Train Loss: 0.1626 Train IoU: 0.7375 Train Dice: 0.8376 Val Loss: 0.1582 Val IoU: 0.7429 Val Dice: 0.8419 Đã lưu model Dice Loss tốt nhất!	Epoch [6/10] Train Loss: 0.1505 Train IoU: 0.7544 Train Dice: 0.8490 Val Loss: 0.1617 Val IoU: 0.7400 Val Dice: 0.8380 Epoch [7/10] Train Loss: 0.1424 Train IoU: 0.7669 Train Dice: 0.8570 Val Loss: 0.1397 Val IoU: 0.7691 Val Dice: 0.8600 Đã lưu model Dice Loss tốt nhất! Epoch [8/10] Train Loss: 0.1328 Train IoU: 0.7810 Train Dice: 0.8670 Val Loss: 0.1827 Val IoU: 0.7131 Val Dice: 0.8170 Epoch [9/10] Train Loss: 0.1281 Train IoU: 0.7878 Train Dice: 0.8710 Val Loss: 0.1260 Val IoU: 0.7913 Val Dice: 0.8740 Đã lưu model Dice Loss tốt nhất! Epoch [10/10] Train Loss: 0.1238 Train IoU: 0.7945 Train Dice: 0.8760 Val Loss: 0.1845 Val IoU: 0.7145 Val Dice: 0.8150 Best Dice Loss Validation IoU: 0.7913202939806758
--	---

Hình 16: Kết quả thử nghiệm Dice Loss

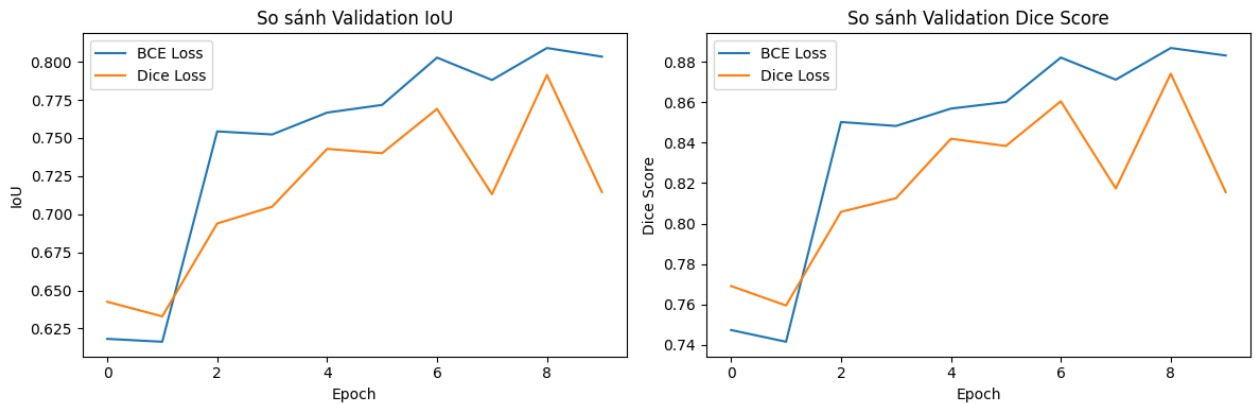
Nhận xét:

Trong thử nghiệm, Dice Loss được huấn luyện trong 10 epoch với cùng learning rate 0.001. Kết quả tốt nhất đạt validation IoU 0.7913 và Dice Score tiêu biểu 0.8740. So với BCEWithLogitsLoss, Dice Loss cho kết quả thấp hơn trong cấu hình hiện tại. Tuy nhiên, điều này không có nghĩa Dice Loss luôn kém hơn, mà cho thấy BCEWithLogitsLoss hội tụ ổn định hơn với số epoch và thiết lập đang sử dụng.

Bảng 7: So sánh kết quả giữa BCEWithLogitsLoss và Dice Loss

Cấu hình	Số epoch	Best Val IoU	Val Dice tốt nhất/tiêu biểu	Nhận xét
BCEWithLogitsLoss	20	0.8484	0.9120	Kết quả cao nhất, đường học ổn định sau các epoch giữa.
Dice Loss	10	0.7913	0.8740	Tối ưu trực tiếp overlap nhưng còn

				dao động ở epoch cuối.
--	--	--	--	------------------------



Hình 17: So sánh IoU và Dice Score giữa BCE Loss và Dice Loss

Quan sát biểu đồ so sánh IoU và Dice Score, đường kết quả của BCE Loss nhìn chung nằm cao hơn Dice Loss ở hầu hết các epoch. BCE Loss có xu hướng tăng ổn định sau các epoch đầu, trong khi Dice Loss vẫn cải thiện nhưng dao động mạnh hơn, đặc biệt ở các epoch cuối. Nguyên nhân có thể do Dice Loss tối ưu trực tiếp mức độ chồng lấp giữa predicted mask và ground truth mask, nên nhạy hơn với sự thay đổi của vùng foreground trong từng batch dữ liệu.

Tuy Dice Loss có kết quả thấp hơn trong thử nghiệm này, nhưng không thể kết luận Dice Loss luôn kém hiệu quả. Dice Loss vẫn có ưu điểm trong các bài toán segmentation, đặc biệt khi dữ liệu bị mất cân bằng giữa foreground và background. Tuy nhiên, với cấu hình hiện tại của bài thực hành, BCEWithLogitsLoss là lựa chọn phù hợp hơn vì cho kết quả cao hơn và quá trình học ổn định hơn.

Một hướng cải thiện có thể thực hiện trong các lần thử nghiệm tiếp theo là kết hợp BCEWithLogitsLoss và Dice Loss. Cách kết hợp này có thể tận dụng được ưu điểm của BCE trong việc phân loại từng pixel ổn định, đồng thời tận dụng khả năng tối ưu mức độ chồng lấp mask của Dice Loss.

4.5 Bài 9: Tính IoU trên tập validation

Sau mỗi epoch, mô hình được đánh giá trên tập validation bằng chỉ số Intersection over Union (IoU). Trong quá trình đánh giá, logits đầu ra của mô hình được đưa qua sigmoid để chuyển thành xác suất. Sau đó, ngưỡng 0.5 được sử dụng để tạo mask nhị phân. Mask dự đoán được so sánh với ground truth mask để tính phần giao và phần hợp.

Kết quả tốt nhất của mô hình U-Net với BCEWithLogitsLoss đạt validation IoU 0.8484. Giá trị này cho thấy phần lớn vùng thú cưng trong ảnh validation đã được mô hình dự đoán trùng khớp với mask thật. Ngoài IoU, Dice Score tốt nhất đạt 0.9120, phản ánh mức độ tương đồng cao giữa predicted mask và ground truth mask.

Bảng 8: Kết quả đánh giá tốt nhất trên tập validation

Mô hình	Loss	Best Val IoU	Val Dice	Epoch tốt nhất
U-Net	BCEWithLogitsLoss	0.8484	0.9120	19

4.6 Bài 10: Hiển thị segmentation mask dự đoán

Sau khi huấn luyện, best model được nạp lại để thực hiện dự đoán trên một số ảnh thuộc tập validation. Kết quả được trực quan hóa gồm ba phần: ảnh gốc, ground truth mask và predicted mask. Cách hiển thị này giúp quan sát trực tiếp khả năng phân đoạn của mô hình, đặc biệt là độ bám sát vùng đối tượng và chất lượng đường biên.

```
model_bce.load_state_dict(  
    torch.load(best_model_path, map_location=device)  
)  
  
model_bce.eval()  
  
def show_predictions(model, dataset, num_samples=5):  
  
    plt.figure(figsize=(12, num_samples * 4))  
  
    for i in range(min(num_samples, len(dataset))):
```

```

image, mask = dataset[i]

with torch.no_grad():

    output = model(
        image.unsqueeze(0).to(device)
    )

    pred = torch.sigmoid(output)

    pred = pred.squeeze().cpu().numpy()

    pred = (pred > 0.5).astype(np.uint8)

image_np = image.permute(1, 2, 0).numpy()
mask_np = mask.squeeze().numpy()

plt.subplot(num_samples, 3, i * 3 + 1)
plt.imshow(image_np)
plt.title("Original Image")
plt.axis("off")

plt.subplot(num_samples, 3, i * 3 + 2)
plt.imshow(mask_np, cmap="gray")
plt.title("Ground Truth")
plt.axis("off")

plt.subplot(num_samples, 3, i * 3 + 3)
plt.imshow(pred, cmap="gray")
plt.title("Prediction")
plt.axis("off")

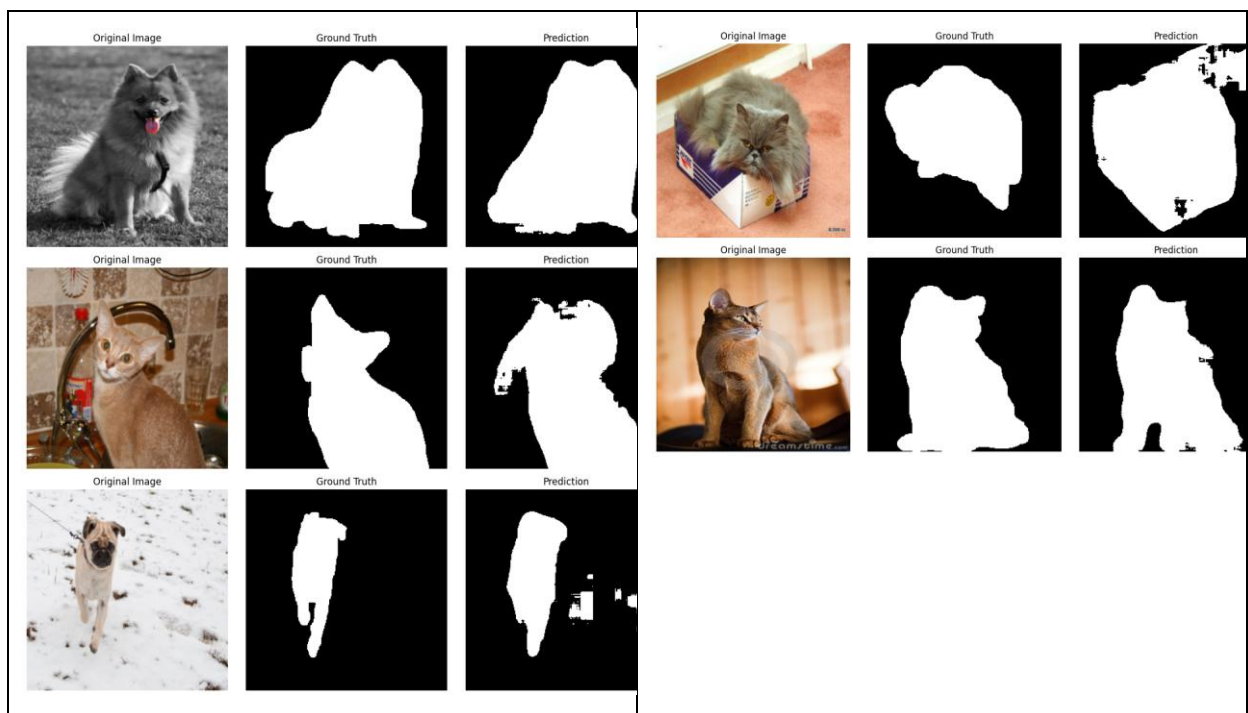
plt.tight_layout()

plt.savefig(
    os.path.join(WORK_DIR, "prediction_results.png"),
    dpi=300,
    bbox_inches="tight"
)

```

```
plt.show()

# Hiển thị trên tập validation
show_predictions(
    model_bce,
    val_dataset,
    num_samples=5
)
```



Hình 18: Kết quả dự đoán segmentation trên tập validation

Quan sát các ảnh dự đoán cho thấy mô hình có khả năng tách vùng thú cưng khá tốt trong những trường hợp đối tượng rõ, nền đơn giản và có độ tương phản cao. Tuy nhiên, ở một số ảnh có nền phức tạp hoặc vùng biên mảnh, predicted mask vẫn có hiện tượng mất chi tiết hoặc dự đoán thừa một phần background.

4.7 Bài 11: So sánh kết quả khi thay đổi learning rate

Learning rate là một siêu tham số quan trọng ảnh hưởng trực tiếp đến tốc độ hội tụ và độ ổn định của quá trình huấn luyện. Trong bài thực hành, ba giá trị learning rate được thử nghiệm nhanh trong 5 epoch gồm 0.01, 0.001 và 0.0001.

```

def quick_train_lr(lr, epochs=5):
    temp_model = UNet().to(device)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(temp_model.parameters(), lr=lr)

    best_val_iou = 0

    for epoch in range(epochs):
        print(f"LR={lr} | Epoch {epoch+1}/{epochs}")

        train_loss, train_iou, train_dice_value = train_one_epoch(
            temp_model, train_loader, optimizer, criterion
        )

        val_loss, val_iou, val_dice_value = validate_one_epoch(
            temp_model, val_loader, criterion
        )

        best_val_iou = max(best_val_iou, val_iou)

        print(f"Val IoU: {val_iou:.4f} | Val Dice: {val_dice_value:.4f}")

    return best_val_iou

learning_rates = [0.01, 0.001, 0.0001]
lr_results = {}

for lr in learning_rates:
    print("\n=====")
    print("Training với learning rate:", lr)
    print("=====")

    lr_results[lr] = quick_train_lr(lr, epochs=5)

print("Kết quả Learning Rate:")
for lr, iou in lr_results.items():
    print(f"LR = {lr}: Best Val IoU = {iou:.4f}")

```

<pre> ===== Training với learning rate: 0.01 ===== LR=0.01 Epoch 1/5 Val IoU: 0.3919 Val Dice: 0.5318 LR=0.01 Epoch 2/5 Val IoU: 0.6782 Val Dice: 0.7960 LR=0.01 Epoch 3/5 Val IoU: 0.7176 Val Dice: 0.8236 LR=0.01 Epoch 4/5 Val IoU: 0.7024 Val Dice: 0.8144 LR=0.01 Epoch 5/5 Val IoU: 0.7542 Val Dice: 0.8498 </pre>	<pre> ===== Training với learning rate: 0.001 ===== LR=0.001 Epoch 1/5 Val IoU: 0.5858 Val Dice: 0.7203 LR=0.001 Epoch 2/5 Val IoU: 0.6992 Val Dice: 0.8086 LR=0.001 Epoch 3/5 Val IoU: 0.7483 Val Dice: 0.8452 LR=0.001 Epoch 4/5 Val IoU: 0.7783 Val Dice: 0.8670 LR=0.001 Epoch 5/5 Val IoU: 0.8111 Val Dice: 0.8885 </pre>	<pre> ===== Training với learning rate: 0.0001 ===== LR=0.0001 Epoch 1/5 Val IoU: 0.7434 Val Dice: 0.8421 LR=0.0001 Epoch 2/5 Val IoU: 0.7753 Val Dice: 0.8627 LR=0.0001 Epoch 3/5 Val IoU: 0.7875 Val Dice: 0.8715 LR=0.0001 Epoch 4/5 Val IoU: 0.8038 Val Dice: 0.8828 LR=0.0001 Epoch 5/5 Val IoU: 0.8134 Val Dice: 0.8893 Kết quả Learning Rate: LR = 0.01: Best Val IoU = 0.7542 LR = 0.001: Best Val IoU = 0.8111 LR = 0.0001: Best Val IoU = 0.8134 </pre>
---	---	--

Hình 19: Kết quả khi thay đổi learning rate

Bảng 9: Ảnh hưởng của learning rate đến kết quả validation

Learning rate	Best Val IoU	Diễn giải
0.01	0.7542	Bước cập nhật lớn, dễ dao động, hội tụ ban đầu chưa ổn định.
0.001	0.8111	Cân bằng giữa tốc độ học và độ ổn định, phù hợp với cấu hình chính.
0.0001	0.8134	Ổn định trong thử nghiệm ngắn, nhưng có thể cần nhiều epoch hơn để khai thác hết.

Kết quả cho thấy learning rate 0.01 có Best Val IoU thấp nhất do bước cập nhật quá lớn, làm quá trình học dễ dao động. Hai giá trị 0.001 và 0.0001 cho kết quả tốt hơn trong thử nghiệm ngắn. Trong cấu hình huấn luyện chính 20 epoch, learning rate 0.001 được chọn vì có tốc độ hội tụ nhanh và đạt kết quả validation cao.

4.8 Bài 12: Thử nghiệm các batch size khác nhau

Batch size quyết định số lượng mẫu được xử lý trong mỗi lần cập nhật trọng số. Batch size nhỏ thường tạo ra nhiều lần cập nhật hơn trong một epoch, nhưng gradient

có thể nhiều hơn. Ngược lại, batch size lớn giúp gradient ổn định hơn nhưng yêu cầu nhiều bộ nhớ GPU hơn và số lần cập nhật trong mỗi epoch ít hơn.

<pre>===== Training với batch size: 4 ===== Batch Size=4 Epoch 1/5 Val IoU: 0.5756 Val Dice: 0.7117 Batch Size=4 Epoch 2/5 Val IoU: 0.7166 Val Dice: 0.8239 Batch Size=4 Epoch 3/5 Val IoU: 0.7069 Val Dice: 0.8161 Batch Size=4 Epoch 4/5 Val IoU: 0.7733 Val Dice: 0.8628 Batch Size=4 Epoch 5/5 Val IoU: 0.7893 Val Dice: 0.8744</pre>	<pre>===== Training với batch size: 8 ===== Batch Size=8 Epoch 1/5 Val IoU: 0.6394 Val Dice: 0.7669 Batch Size=8 Epoch 2/5 Val IoU: 0.6932 Val Dice: 0.8070 Batch Size=8 Epoch 3/5 Val IoU: 0.7582 Val Dice: 0.8525 Batch Size=8 Epoch 4/5 Val IoU: 0.7657 Val Dice: 0.8576 Batch Size=8 Epoch 5/5 Val IoU: 0.7633 Val Dice: 0.8542</pre>	<pre>===== Training với batch size: 16 ===== Batch Size=16 Epoch 1/5 Val IoU: 0.6076 Val Dice: 0.7353 Batch Size=16 Epoch 2/5 Val IoU: 0.7120 Val Dice: 0.8204 Batch Size=16 Epoch 3/5 Val IoU: 0.7449 Val Dice: 0.8431 Batch Size=16 Epoch 4/5 Val IoU: 0.7137 Val Dice: 0.8154 Batch Size=16 Epoch 5/5 Val IoU: 0.7700 Val Dice: 0.8567 Kết quả Batch Size: Batch Size = 4: Best Val IoU = 0.7893 Batch Size = 8: Best Val IoU = 0.7657 Batch Size = 16: Best Val IoU = 0.7700</pre>
--	--	---

Hình 20: Kết quả khi thay đổi batchsize

Bảng 10: Ảnh hưởng của batch size đến kết quả validation

Batch size	Best Val IoU	Diễn giải
4	0.7893	Tốt nhất trong thử nghiệm 5 epoch; cập nhật trọng số thường xuyên hơn.
8	0.7657	Cấu hình chính, cân bằng giữa bộ nhớ GPU và tốc độ huấn luyện.
16	0.7700	Gradient ổn định hơn nhưng số bước cập nhật ít hơn trong mỗi epoch.

Trong thử nghiệm 5 epoch, batch size 4 đạt Best Val IoU cao nhất. Tuy nhiên, batch size 8 vẫn được lựa chọn cho cấu hình chính vì phù hợp với bộ nhớ GPU, tốc độ huấn luyện và tính ổn định khi chạy nhiều epoch hơn.

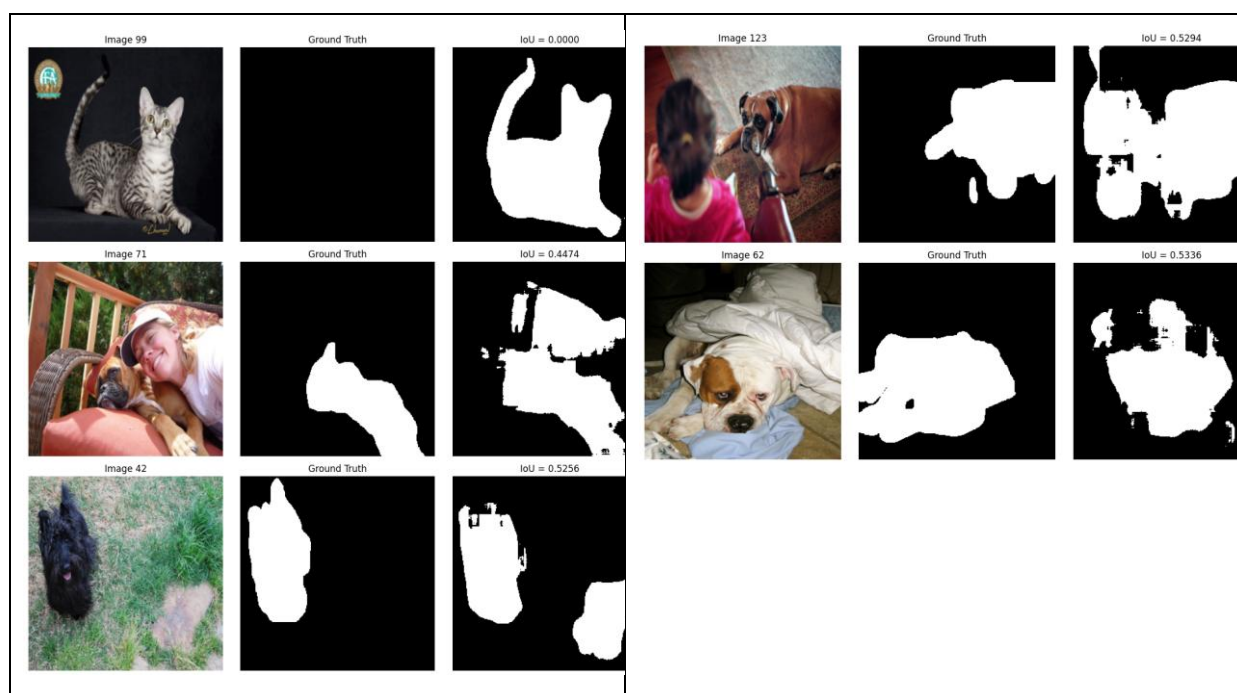
4.9 Bài 13: Phân tích các trường hợp mô hình dự đoán sai

Để đánh giá sâu hơn hạn chế của mô hình, notebook tìm các mẫu có IoU thấp trong tập validation. Các mẫu này giúp chỉ ra những trường hợp mà mô hình phân đoạn chưa

tốt, ví dụ như bỏ sót foreground, dự đoán thừa background hoặc xác định sai đường biên đối tượng.

Bảng 11: Một số mẫu validation có IoU thấp

Chỉ số mẫu	IoU	Nhận xét
99	≈ 0.0000	Dự đoán thất bại gần như hoàn toàn; cần quan sát ảnh để xác định nguyên nhân cụ thể.
71	0.4474	Mask chỉ khớp một phần, có khả năng mất vùng biên hoặc nhầm nền.
42	0.5256	Kết quả trung bình, vùng foreground chưa được bao phủ đầy đủ.
123	0.5294	Dự đoán có overlap nhưng còn sai đáng kể.
62	0.5336	Mức chồng lấp thấp, cần cải thiện biên và chi tiết nhỏ.



Hình 21: Các trường hợp dự đoán có IoU thấp

Các lỗi thường xuất hiện khi ảnh có nền phức tạp, màu nền gần giống đối tượng, đối tượng bị che khuất hoặc có chi tiết nhỏ như tai, chân, lông và đuôi. Ngoài

ra, việc resize ảnh về 256×256 có thể làm mất một số chi tiết biên mảnh. Mask gốc của Oxford-IIIT Pet có vùng boundary riêng, khi gộp boundary vào foreground có thể làm vùng biên dày hơn và khó dự đoán đồng nhất.

4.10 Bài 14: Đề xuất cách cải thiện mô hình

Từ các kết quả thực nghiệm và các mẫu dự đoán sai, có thể đề xuất một số hướng cải thiện mô hình như sau:

- Bổ sung data augmentation như random horizontal flip, random crop, color jitter hoặc rotation nhẹ để tăng khả năng tổng quát hóa của mô hình.
- Thử nghiệm hàm loss kết hợp BCE + Dice Loss để tận dụng độ ổn định của BCE và khả năng tối ưu overlap của Dice Loss.
- Tối ưu threshold trên tập validation thay vì dùng cố định ngưỡng 0.5 cho mọi ảnh.
- Sử dụng encoder pretrained như ResNet hoặc EfficientNet để tăng khả năng trích xuất đặc trưng.
- Tăng số epoch kết hợp early stopping để tránh overfitting và lưu lại mô hình tốt nhất.
- Đánh giá riêng các nhóm ảnh khó, ví dụ ảnh nền phức tạp, đối tượng nhỏ hoặc vùng biên không rõ.

4.11 Bài 15: Thử nghiệm một kiến trúc segmentation khác

Theo yêu cầu mở rộng của bài thực hành, ngoài mô hình U-Net, bài thực hành tiến hành thử nghiệm thêm một kiến trúc segmentation khác là **SegNet**. SegNet cũng là một mô hình dạng encoder-decoder, được sử dụng phổ biến trong các bài toán semantic segmentation. Mục tiêu của phần này là đánh giá hiệu quả của SegNet trên cùng bộ dữ liệu Oxford-IIIT Pet và so sánh kết quả với mô hình U-Net đã huấn luyện trước đó.

```
EPOCHS_SEGNET = 20
LR_SEGNET = 0.001

criterion_segnet = nn.BCEWithLogitsLoss()
optimizer_segnet = optim.Adam(model_segnet.parameters(), lr=LR_SEGNET)

history_segnet = {
    "train_loss": [],
```

```

        "val_loss": [],
        "train_iou": [],
        "val_iou": [],
        "train_dice": [],
        "val_dice": []
    }

best_iou_segnet = 0.0
best_segnet_path = os.path.join(WORK_DIR, "best_segnet_mini.pth")

for epoch in range(EPOCHS_SEGNET):
    print(f"\nEpoch [{epoch+1}/{EPOCHS_SEGNET}]")

    train_loss, train_iou, train_dice = train_one_epoch(
        model_segnet,
        train_loader,
        optimizer_segnet,
        criterion_segnet
    )

    val_loss, val_iou, val_dice = validate_one_epoch(
        model_segnet,
        val_loader,
        criterion_segnet
    )

    history_segnet["train_loss"].append(train_loss)
    history_segnet["val_loss"].append(val_loss)
    history_segnet["train_iou"].append(train_iou)
    history_segnet["val_iou"].append(val_iou)
    history_segnet["train_dice"].append(train_dice)
    history_segnet["val_dice"].append(val_dice)

    print(f"Train Loss: {train_loss:.4f} | Train IoU: {train_iou:.4f} | Train Dice: {train_dice:.4f}")
    print(f"Val Loss: {val_loss:.4f} | Val IoU: {val_iou:.4f} | Val Dice: {val_dice:.4f}")

    if val_iou > best_iou_segnet:
        best_iou_segnet = val_iou
        torch.save(model_segnet.state_dict(), best_segnet_path)
        print("Đã lưu SegNet tốt nhất!")

print("Best SegNet Validation IoU:", best_iou_segnet)

```

Mô hình SegNet được huấn luyện trong 20 epoch với cùng tập dữ liệu, cùng kích thước ảnh đầu vào 256×256 và được đánh giá bằng hai chỉ số chính là IoU và Dice Score. Trong quá trình huấn luyện, mô hình được lưu lại khi validation IoU đạt giá trị tốt hơn các epoch trước đó.

Kết quả:

Epoch [1/20] Train Loss: 0.5226 Train IoU: 0.5222 Train Dice: 0.6684 Val Loss: 0.4869 Val IoU: 0.5354 Val Dice: 0.6818 Đã lưu SegNet tốt nhất! Epoch [2/20] Train Loss: 0.4750 Train IoU: 0.5714 Train Dice: 0.7125 Val Loss: 0.4667 Val IoU: 0.5496 Val Dice: 0.6953 Đã lưu SegNet tốt nhất! Epoch [3/20] Train Loss: 0.4544 Train IoU: 0.5899 Train Dice: 0.7280 Val Loss: 0.4858 Val IoU: 0.5830 Val Dice: 0.7207 Đã lưu SegNet tốt nhất! Epoch [4/20] Train Loss: 0.4389 Train IoU: 0.6044 Train Dice: 0.7398 Val Loss: 0.4799 Val IoU: 0.4894 Val Dice: 0.6388 Epoch [5/20] Train Loss: 0.4257 Train IoU: 0.6168 Train Dice: 0.7496 Val Loss: 0.4477 Val IoU: 0.6003 Val Dice: 0.7358 Đã lưu SegNet tốt nhất!	Epoch [6/20] Train Loss: 0.4173 Train IoU: 0.6245 Train Dice: 0.7557 Val Loss: 0.4450 Val IoU: 0.6267 Val Dice: 0.7545 Đã lưu SegNet tốt nhất! Epoch [7/20] Train Loss: 0.4094 Train IoU: 0.6312 Train Dice: 0.7606 Val Loss: 0.4171 Val IoU: 0.5953 Val Dice: 0.7329 Epoch [8/20] Train Loss: 0.4049 Train IoU: 0.6348 Train Dice: 0.7637 Val Loss: 0.3992 Val IoU: 0.6385 Val Dice: 0.7677 Đã lưu SegNet tốt nhất! Epoch [9/20] Train Loss: 0.4022 Train IoU: 0.6368 Train Dice: 0.7651 Val Loss: 0.3977 Val IoU: 0.6301 Val Dice: 0.7592 Epoch [10/20] Train Loss: 0.3947 Train IoU: 0.6437 Train Dice: 0.7707 Val Loss: 0.4394 Val IoU: 0.6355 Val Dice: 0.7610
Epoch [11/20] Train Loss: 0.3915 Train IoU: 0.6466 Train Dice: 0.7728 Val Loss: 0.3978 Val IoU: 0.6532 Val Dice: 0.7765 Đã lưu SegNet tốt nhất! Epoch [12/20] Train Loss: 0.3876 Train IoU: 0.6499 Train Dice: 0.7751 Val Loss: 0.4134 Val IoU: 0.5639 Val Dice: 0.7076 Epoch [13/20] Train Loss: 0.3868 Train IoU: 0.6504 Train Dice: 0.7756 Val Loss: 0.4126 Val IoU: 0.6454 Val Dice: 0.7693 Epoch [14/20] Train Loss: 0.3848 Train IoU: 0.6531 Train Dice: 0.7776 Val Loss: 0.4106 Val IoU: 0.6545 Val Dice: 0.7764 Đã lưu SegNet tốt nhất! Epoch [15/20] Train Loss: 0.3804 Train IoU: 0.6563 Train Dice: 0.7801 Val Loss: 0.4203 Val IoU: 0.6458 Val Dice: 0.7699	Epoch [16/20] Train Loss: 0.3756 Train IoU: 0.6608 Train Dice: 0.7835 Val Loss: 0.3897 Val IoU: 0.6625 Val Dice: 0.7829 Đã lưu SegNet tốt nhất! Epoch [17/20] Train Loss: 0.3742 Train IoU: 0.6633 Train Dice: 0.7853 Val Loss: 0.3704 Val IoU: 0.6535 Val Dice: 0.7783 Epoch [18/20] Train Loss: 0.3756 Train IoU: 0.6613 Train Dice: 0.7840 Val Loss: 0.3790 Val IoU: 0.6410 Val Dice: 0.7700 Epoch [19/20] Train Loss: 0.3709 Train IoU: 0.6647 Train Dice: 0.7863 Val Loss: 0.3894 Val IoU: 0.6694 Val Dice: 0.7884 Đã lưu SegNet tốt nhất! Epoch [20/20] Train Loss: 0.3687 Train IoU: 0.6669 Train Dice: 0.7881 Val Loss: 0.4114 Val IoU: 0.6596 Val Dice: 0.7804 Best SegNet Validation IoU: 0.6694370339045653

Hình 22: Kết quả huấn luyện SegNet

Kết quả huấn luyện cho thấy SegNet có khả năng học được đặc trưng phân đoạn, thể hiện qua việc Train Loss giảm từ 0.5226 ở epoch 1 xuống còn 0.3687 ở epoch 20. Đồng thời, Train IoU tăng từ 0.5222 lên 0.6669, và Train Dice tăng từ 0.6684 lên 0.7881. Điều này cho thấy mô hình có cải thiện dần qua các epoch trên tập huấn luyện.

Trên tập validation, kết quả tốt nhất của SegNet đạt được tại epoch 19 với Val IoU = 0.6694 và Val Dice = 0.7884. Tuy nhiên, kết quả validation có sự dao động giữa các epoch. Ví dụ, Val IoU đạt 0.6625 ở epoch 16, giảm xuống 0.6410 ở epoch 18, sau đó tăng lên 0.6694 ở epoch 19. Điều này cho thấy mô hình vẫn chưa thật sự ổn định trên tập validation.

Bảng 12: So sánh kết quả giữa U-Net và SegNet

Mô hình	Số epoch	Loss	Best Val IoU	Best Val Dice	Nhận xét
U-Net	20	BCEWithLogitsLoss	0.8484	0.9120	Kết quả tốt nhất trong bài, mask dự đoán bám sát vùng thú cưng hơn.
SegNet	20	BCEWithLogitsLoss	0.6694	0.7884	Có học được đặc trưng phân đoạn nhưng kết quả thấp hơn U-Net.

Nhận xét:

Qua bảng so sánh, có thể thấy mô hình U-Net cho kết quả vượt trội hơn SegNet trong bài toán phân đoạn ảnh thú cưng. U-Net đạt **Best Val IoU = 0.8484** và **Best Val Dice = 0.9120**, trong khi SegNet chỉ đạt **Best Val IoU = 0.6694** và **Best Val Dice = 0.7884**. Sự chênh lệch này cho thấy U-Net dự đoán vùng đối tượng chính xác hơn và có mức độ chồng lấp với ground truth mask cao hơn.

Nguyên nhân SegNet cho kết quả thấp hơn có thể đến từ việc kiến trúc này khôi phục độ phân giải ở decoder nhưng không tận dụng skip connection mạnh như U-Net. Trong khi đó, U-Net truyền trực tiếp đặc trưng từ encoder sang decoder, giúp giữ lại thông tin vị trí và đường biên của đối tượng tốt hơn. Với bài toán phân đoạn ảnh thú cưng, các chi tiết như tai, chân, lông và biên cơ thể rất quan trọng, nên U-Net phù hợp hơn.

Nhìn chung, SegNet đã hoàn thành vai trò là mô hình thử nghiệm bổ sung cho Bài 15. Kết quả cho thấy SegNet có thể thực hiện semantic segmentation, nhưng trong cấu hình thực nghiệm hiện tại, U-Net vẫn là mô hình hiệu quả hơn và được chọn làm mô hình chính cho bài toán phân đoạn ảnh Oxford-IIIT Pet.

4.12 Tổng hợp kết quả thực nghiệm

Bảng 13: Tổng hợp kết quả thực nghiệm chính

Nội dung thử nghiệm	Kết quả chính	Nhận xét
U-Net với BCE Loss	Best Val IoU = 0.8484; Val Dice = 0.9120	Cấu hình tốt nhất trong các thử nghiệm đã chạy.
Dice Loss	Best Val IoU = 0.7913	Tối ưu overlap nhưng chưa ổn định bằng BCE trong cấu hình hiện tại.
Learning rate	0.001 và 0.0001 tốt hơn 0.01	Learning rate quá lớn làm quá trình học dễ dao động.
Batch size	Batch size 4 tốt nhất trong thử nghiệm 5 epoch	Batch size 8 vẫn cân bằng hơn cho cấu hình chính.
Mẫu IoU thấp	Một số mẫu thấp do nền phức tạp, biên khó	Cần cải thiện bằng augmentation, threshold tuning hoặc encoder pretrained.

Nhìn chung, mô hình U-Net với BCEWithLogitsLoss là cấu hình đạt kết quả tốt nhất trong các thử nghiệm hiện tại. Mô hình có khả năng phân đoạn vùng thú cưng khá chính xác, đặc biệt với các ảnh có đối tượng rõ và nền ít phức tạp. Tuy nhiên, mô hình vẫn gặp khó khăn ở những ảnh có nền phức tạp, đối tượng nhỏ hoặc biên không rõ. Các hướng cải thiện như data augmentation, loss kết hợp BCE + Dice, tối ưu threshold và thử nghiệm DeepLabV3 có thể giúp nâng cao chất lượng phân đoạn trong các lần thực nghiệm tiếp theo.

5. PHÂN TÍCH VÀ THẢO LUẬN

5.1 Nhận xét về kết quả đạt được

Kết quả tốt nhất của mô hình BCE đạt validation IoU 0.8484 và validation Dice 0.9120. Đây là mức kết quả khá tốt đối với một U-Net tự xây dựng từ đầu trên ảnh 256x256. Đường train IoU tăng từ 0.5729 lên 0.8812, còn validation IoU tăng từ 0.6183 lên vùng 0.84, cho thấy mô hình học được biểu diễn phân đoạn có khả năng tổng quát hóa tương đối tốt.

Tuy nhiên, có dấu hiệu dao động ở validation loss và validation IoU, ví dụ epoch 17 giảm xuống 0.7696 rồi tăng lại ở các epoch sau. Điều này có thể đến từ tính đa dạng của tập validation, độ khó của vùng biên, hoặc việc dùng một ngưỡng cố định 0.5 cho mọi ảnh.

5.2 So sánh loss function

BCEWithLogitsLoss hoạt động tốt vì bài toán đã được chuyển thành phân đoạn nhị phân và mỗi pixel được xem như một quan sát nhị phân. Dice Loss có ưu điểm là tối ưu trực tiếp overlap, nhưng trong notebook chỉ huấn luyện 10 epoch và có dao động ở epoch cuối. Nếu tiếp tục thử nghiệm, có thể dùng tổ hợp BCE + Dice Loss để tận dụng cả độ ổn định của BCE và khả năng tối ưu vùng chồng lấp của Dice.

5.3 Ảnh hưởng của siêu tham số

Learning rate 0.01 cho thấy kết quả đầu thấp vì bước cập nhật quá lớn, trong khi 0.001 và 0.0001 đều cho kết quả tốt hơn. Với training dài 20 epoch, 0.001 là lựa chọn hợp lý vì hội tụ nhanh và đạt best IoU cao. Batch size 4 có kết quả tốt nhất trong thử nghiệm ngắn, nhưng batch size 8 vẫn là lựa chọn cân bằng vì tận dụng GPU tốt hơn và được dùng cho cấu hình chính.

5.4 Nguyên nhân lỗi dự đoán

Một số ảnh có nền phức tạp hoặc màu nền gần giống đối tượng làm mô hình khó tách foreground/background.

Các vùng lông, tai, chân hoặc biên mảnh dễ bị mất khi resize về 256x256.

Mask ground truth của Oxford-IIIT Pet có vùng boundary riêng; việc gộp boundary vào foreground có thể làm biên dày hơn và khó dự đoán đồng nhất.

U-Net tự xây dựng chưa dùng encoder được pretrain nên khả năng trích xuất đặc trưng chưa mạnh như các mô hình segmentation hiện đại.

Ngưỡng 0.5 cố định có thể chưa tối ưu cho mọi ảnh; một số ảnh cần threshold khác để cân bằng precision và recall của mask.

5.5 Ưu điểm và hạn chế của mô hình

Nhóm	Nội dung
Ưu điểm	Pipeline rõ ràng, kết quả IoU/Dice tốt, mô hình U-Net giữ được thông tin không gian nhờ skip connection.
Ưu điểm	Có checkpoint, best model và trực quan hóa kết quả nên dễ theo dõi quá trình huấn luyện.
Hạn chế	Chưa dùng data augmentation, chưa thử encoder pretrained và chưa tối ưu threshold theo validation set.
Hạn chế	Thử nghiệm Dice Loss, learning rate và batch size mới ở mức nhanh, chưa đánh giá nhiều lần để giảm ảnh hưởng ngẫu nhiên.

6. KẾT LUẬN VÀ HƯỚNG CẢI THIẾN

Bài thực hành đã hoàn thành pipeline semantic segmentation bằng PyTorch: đọc dataset, tiền xử lý ảnh/mask, xây dựng Dataset class, tạo DataLoader, xây dựng U-Net, huấn luyện với BCE Loss và Dice Loss, đánh giá bằng IoU/Dice Score, trực quan hóa predicted mask và phân tích các mẫu lỗi. Kết quả tốt nhất đạt validation IoU 0.8484, cho thấy mô hình U-Net có khả năng phân đoạn vùng thú cưng khá chính xác.

Hướng cải thiện tiếp theo gồm: bổ sung data augmentation như random flip, color jitter và random crop; dùng loss kết hợp BCE + Dice; thử threshold tối ưu trên validation set; sử dụng encoder pretrained như ResNet/EfficientNet; thử các kiến trúc DeepLabV3 hoặc U-Net++; tăng số epoch kèm early stopping; và đánh giá riêng các nhóm ảnh khó để hiểu rõ hơn lỗi của mô hình.